

---

# 复习C语言

## 用户自己建立数据类型

例

```
struct student
{
    int num;
    char name[20];
    char sex;
    int age;
    int score;
    char addr[30];
}; //先自己建立一个结构体类型
struct student student_1;
//然后根据这个类型，定义一个变量
```

# 定义结构体类型变量

例

```
struct student
{
    int num;
    char name[20];
    char sex;
    int age;
    float score;
    char addr[30];
};
struct student stu1,stu2;
```

```
struct  结构体名
{
    类型标识符  成员名;
    类型标识符  成员名;
    .....
};
struct 结构体名 变量名表列;
```

# 有以下程序

---

```
#include <stdio.h>
void main( )
{
    struct STU
    {
        char name[9];
        char sex;
        int score;
    };
    struct STU a={"Zhaosi",'m',90}
    printf("%s,%c,%d\n", a.name, a.sex, a.score);
}
```

程序的运行结果是 Zhaosi,m,90

---

# 第2章 线性表

2.1 线性表的定义和特点

2.3 线性表的类型定义

2.4 线性表的顺序表示和实现

2.5 线性表的链式表示和实现

2.6 顺序表和链表的比较

2.7 线性表的应用

# 数据结构 (非数值问题)

数据的逻辑结构

线性结构

线性表  
栈、队列  
串  
数组、广义表

非线性结构

树  
图

数据的存储结构

顺序存储 (数组)

链式存储 (指针)

数据的操作：插入、删除、修改、查找、排序

## 2.1 线性表的定义和特点(P18)

线性结构：<sup>开始元素</sup> $(\textcircled{a_1}, a_2, \dots, \textcircled{a_n})$ <sup>终端元素</sup>的特点：

- ①存在唯一的被称作“第一个”的数据元素
- ②存在唯一的被称作“最后一个”的数据元素
- ③除第一个元素之外，结构中的每个数据元素都有且仅有一个直接前驱；
- ④除最后一个元素之外，结构中的每个数据元素都有且仅有一个直接后继。

线性结构包括线性表、栈、队列、字符串、数组等，其中，最典型、最常用的是



线性表

## 例1 分析26个英文字母组成的英文表

( A, B, C, D, ..... , Z)

数据元素都是字母; 元素间关系是线性的

## 例2 分析学生情况登记表

|     | 学号        | 姓名  | 性别 | 年龄 | 班级       |
|-----|-----------|-----|----|----|----------|
| 元素1 | 041810205 | 于春梅 | 女  | 18 | 04级计算机1班 |
| 元素2 | 041810260 | 何仕鹏 | 男  | 20 | 04级计算机2班 |
| 元素3 | 041810284 | 王 爽 | 女  | 19 | 04级计算机3班 |
|     | :         | :   | :  | :  | :        |
| 元素n | 041810360 | 王亚武 | 男  | 18 | 04级计算机4班 |

数据元素都是记录; 元素间关系是线性的

**线性表：具有相同数据类型的 $n$ 个数据元素的有限序列。**

$n \geq 0$  当 $n=0$ 为空表

线性表 $L=(a_1, a_2, \dots, a_n)$ ，下列说法正确的是（ ）

- A. 每个元素都有一个直接前驱和一个直接后继
- B. 线性表中至少有一个元素
- C. 表中诸元素的排列必须是由小到大或由大到小
- D. 除第一个和最后一个元素外，其余每个元素都有一个且仅有一个直接前驱和直接后继。

答案：D，A选项开始元素无直接前驱，终端元素无直接后继；B选项：当 $n=0$ 时 $L$ 是空表；C选项： $a_i$ 的下标 $i$ 表示在线性表中的序号或位置，不指代实际大小。

线性表是具有 $n$ 个 ( ) 的有限序列。

A. 数据表

B. 字符

C. 数据元素

D. 数据项

答案: C

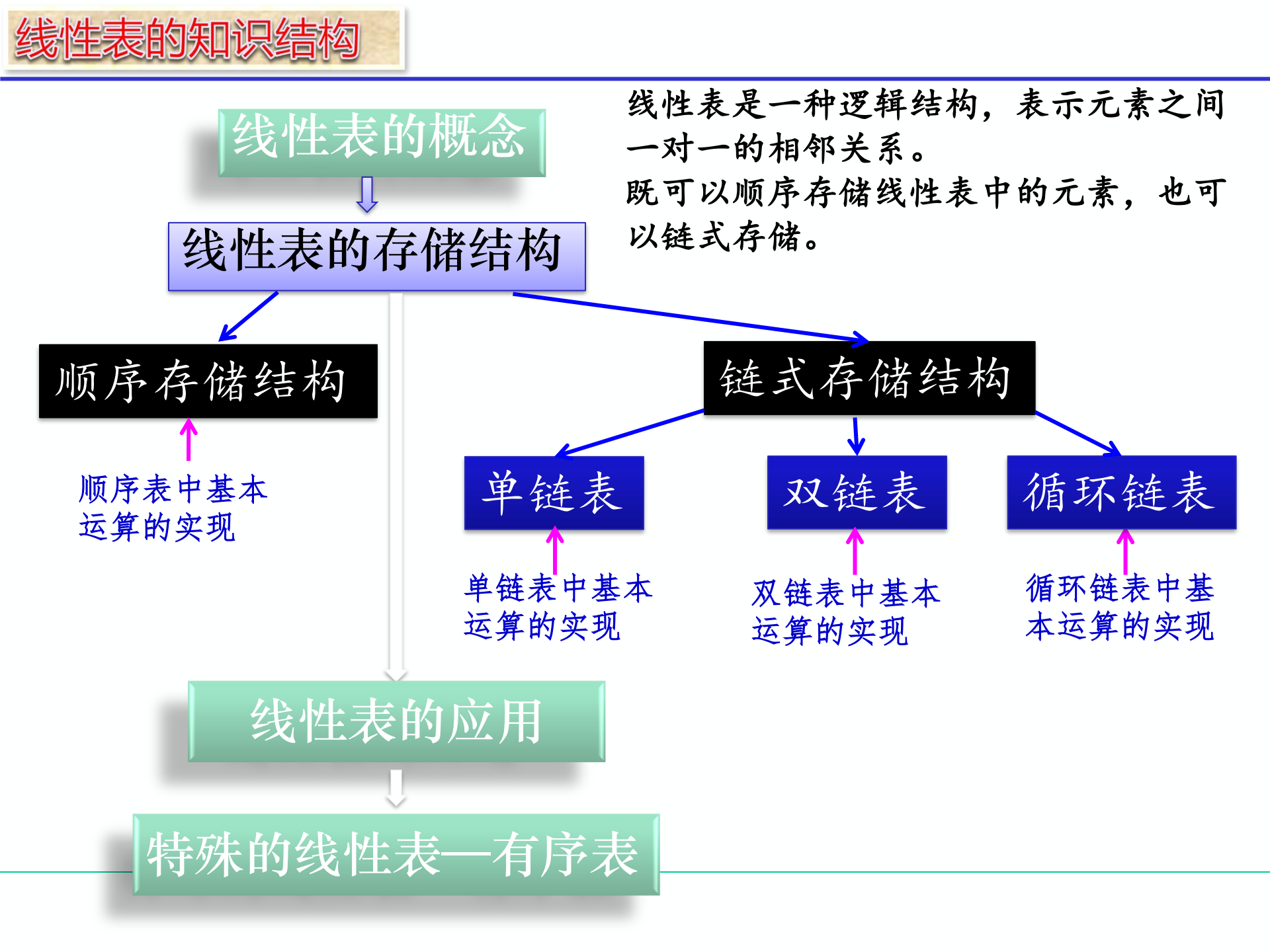
下列两种情况是否是线性表?

①由100个字符组成的序列

✓

②所有整数

✗



## 2.3 线性表的抽象数据类型定义(P21)

ADT List {

数据对象:

$D = \{ a_i \mid a_i \in \text{ElemSet}, i=1,2,\dots,n, n \geq 0 \}$

{称  $n$  为线性表的表长; 称  $n=0$  时的线性表为空表。}

数据关系:

$R1 = \{ \langle a_{i-1}, a_i \rangle \mid a_{i-1}, a_i \in D, i=2,\dots,n \}$

{设线性表为  $(a_1, a_2, \dots, a_i, \dots, a_n)$ , 称  $i$  为  $a_i$  在线性表中的序号。}

基本操作:

初始化操作    插入操作    求表长

销毁操作      删除操作    清空

.....

} ADT List

## 2.4 线性表的顺序表示和实现(P22)



**顺序存储定义：**把逻辑上相邻的数据元素存储在物理位置上相邻的存储单元中。

**简言之，逻辑上相邻，物理上也相邻**

例如：线性表 (1, 2, 3, 4, 5, 6) 的存储结构：

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|

依次存储，相对位置一致，地址连续——  
中间没有空出存储单元。

**是典型的顺序存储结构**

|   |   |  |  |   |   |   |   |
|---|---|--|--|---|---|---|---|
| 1 | 2 |  |  | 3 | 4 | 5 | 6 |
|---|---|--|--|---|---|---|---|

地址不连续——  
中间存在空的存储单元。

**不是线性表的顺序存储结构**

## 2.4 线性表的顺序存储(P22)

$L_0$ 为首元素的起始地址（基地址），  
 $m$ 为每个数据元素需占的存储单元大小

存储地址    存储内容

$L_0$

$L_0+m$

第*i*个元素的起始地址：

$L_0+(i-1)*m$

元素1

元素2

.....

元素*i*

.....

元素*n*

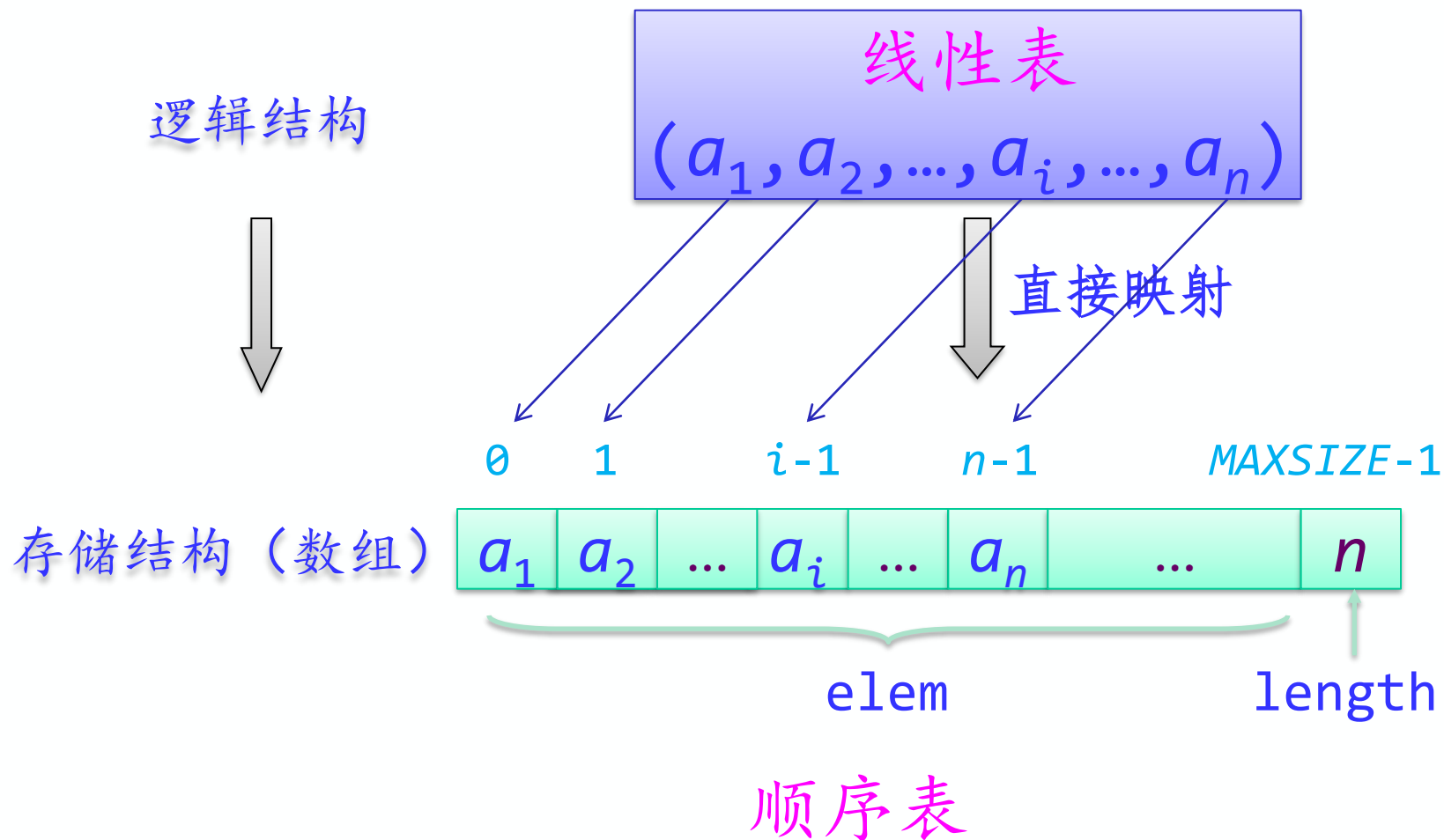
**随机存取特性：**

可以在 $O(1)$ 时间内找到序号为*i*的元素值。

**(优点)**

由于高级程序设计语言中**数组**类型也有随机存取的特性，  
因此通常使用**数组**来描述顺序存储结构。

## 2.4 线性表的顺序存储



**注意：**逻辑序号和物理下标相差1。 P24 说明(3)

顺序表中第一个元素的存储地址是100，每个元素的长度为2，则第5个元素的地址是（ ）。

A. 110                  B. 108                  C. 100                  D. 120

**答案：B**

解释：顺序表中的数据连续存储，所以第5个元素的地址为： $100 + 2 * 4 = 108$ 。

## 2.4 线性表的顺序存储(P23)

|     | 学号        | 姓名  | 性别 | 年龄 | 班级       |
|-----|-----------|-----|----|----|----------|
| 元素1 | 041810205 | 于春梅 | 女  | 18 | 04级计算机1班 |
| 元素2 | 041810260 | 何仕鹏 | 男  | 20 | 04级计算机2班 |
| 元素3 | 041810284 | 王 爽 | 女  | 19 | 04级计算机3班 |
| 元素4 | 041810360 | 王亚武 | 男  | 18 | 04级计算机4班 |
|     | :         | :   | :  | :  | :        |

图1 学籍数据的逻辑结构示意图

|         |         |         |         |       |                |
|---------|---------|---------|---------|-------|----------------|
| elem[0] | elem[1] | elem[2] | elem[3] | ..... | elem[length-1] |
| 元素1     | 元素2     | 元素3     | 元素4     | ..... | 最后一个元素         |

图2 学籍数据的顺序存储结构示意图

# 顺序表的类型定义(P23)

## 数组的定义 (动态分配)

```
#define MAXSIZE 100           //宏定义线性表最大长度
typedef struct
{
    ElemType *elem;           //动态分配的指针, 存储基地址
    int length;               //顺序表的当前长度(元素的个数)
}SqList;                     //结构类型定义为SqList 顺序表
```

Sequence List

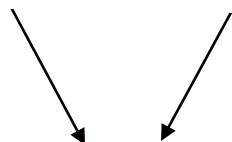
## 数组的定义 (静态分配)

```
#define MAXSIZE 100
typedef struct
{
    ElemType elem[MAXSIZE];
    int length;
}SqList;
```

# 顺序表的类型定义(P23)

可以根据自己的实际需求修改为

char          int



**ElemType \*elem;**

**ElemType elem[ ];**  
//动态分配数组的指针

**Element Type**, 元素的类型, 是数据结构为了说明问题提出的一种数据类型统称, 可以广义、笼统地定义用户想使用的数据类型, 使代码可读性强

也可以提前定义:

**typedef char ElemType;** //假设 *ElemType* 是 *char* 类型

**typedef int ElemType;** //假设 *ElemType* 是 *int* 类型

# 顺序表的类型定义

```
#define MAXSIZE 100           //宏定义表的最大长度100
typedef struct
{
    ElemType *elem;           //动态分配数组空间的指针
    int length;               //顺序表中元素个数
}SqList;                      //顺序表结构类型定义为SqList
```

**SqList \*L;** //定义指针变量L

L→elem //指针变量通过箭头指示顺序表中的数据元素

L→length //当前顺序表的长度

**SqList L;** //定义变量L，L是SqList类型的顺序表

L.elem[i-1] //顺序表中第i个数据元素

L.length //当前顺序表的长度

(P25)

# 图书表的顺序存储结构类型定义(P25)

```
#define MAXSIZE 10000 //图书表可能达到的最大长度
typedef struct          //图书信息定义
{
    char no[20];         //图书ISBN
    char name[50];       //图书名字
    float price;         //图书价格
}Book;                  //结构类型Book，存储图书信息
```

```
typedef struct
{
    Book *elem;          //存储空间的基地址
    int length;          //图书表中当前图书个数
}SqList;                //图书表的顺序存储结构类型为SqList
```

```
SqList L;               //定义变量L，L是SqList类型的顺序表
L.elem[i-1]             //访问表中位置序号为i的图书
```

(P25)

# 算法中用到的预定义常量和类型(P8)

**//函数结果返回的状态代码**

**#define OK 1**

**#define ERROR 0**

**#define OVERFLOW -2 //溢出**

**#define TRUE 1**

**#define FALSE 0**

**//Status是函数返回值的类型，其值是函数结果状态代码**

**typedef int Status; //指示返回值的类型**

**typedef char ElemType; //ElemType指示元素的类型**

# 线性表中基本操作的实现(P22)

引用参数，用于提供输入值或返回操作结果(P8)

- `InitList(&L)` //初始化操作，建立一个空的线性表L
- `DestroyList(&L)` //销毁已存在的线性表L
- `ClearList(&L)` //将已存在的线性表清空（重置为空表）
- `ListEmpty(L)` //若线性表为空，返回true，否则false
- `ListLength(L)` //求表长，返回线性表L的元素个数
- `GetElem(L, i, &e)` //通过e返回线性表L中第i个数据元素的值
- `LocateElem(L, e)` //查找表L中与给定值e相等的元素，若成功  
返回该元素在表中的序号，否则返回0
- `ListInsert(&L, i, e)` //在线性表L中第i个位置插入新元素e
- `ListDelete(&L, i, &e)` //删除线性表L中第i个位置元素，用e返回

# 顺序表中基本操作的实现(P25-26)

## 1. 初始化线性表L

Status InitList\_Sq(SqList &L)

{       **//构造一个空的顺序表L**

    L.elem=new ElemType[MAXSIZE];   **//为顺序表分配空间**

    if(!L.elem)                       **//异常处理语句**

        exit(OVERFLOW);               **//存储分配失败**

    L.length=0;                       **//空表长度为0**

    return OK;

}

!空(0)代表真(1)  
!不空(1)代表假(0)

引用参数：将执行  
结果回传给实参

**new 类型名T[初值]： 申请用于存放T类型对象的内存空间**

## 2. 取值 (获取顺序表中第i个数据元素的值) P26

引用参数:  
通过e返回值

Status GetElem(SqList L, int i, ElemType &e)

{//如果位置正确, 找到就通过引用值返回, i值不合理则直接返回错误

if (i<1 || i>L.length) //判断i值是否合理

return ERROR; //不合理, 返回ERROR

e=L.elem[i-1];

//第i个数据的存储下标是i-1  
逻辑序号和物理下标相差1

return OK;

}

$O(1)$

时间复杂度?  $\longrightarrow$  体现顺序表的随机存取特性

### 3. 按值查找 (查找值为e的数据元素, 返回其**序号**) P26

```
int LocateElem(SqList L, ElemType e) {
```

```
//从线性表L的第一个元素开始, 逐个将数据元素和给定值e进行比较  
找到返回其序号, 查找失败返回0
```

```
for (i=0; i<=L.length-1; i++)
```

```
    if (L.elem[i]==e) return i+1; //查找成功, 返回序号
```

```
return 0; //查找失败, 返回0
```

```
}
```

时间复杂度?   $O(n)$

P27

# 4. 插入 (在顺序表L中第i个位置插入数据元素e) P27

(25, 12, 47, 89, 36, 14)

e=99

99插入

|   |    |
|---|----|
| 1 | 25 |
| 2 | 12 |
| 3 | 47 |
| 4 | 89 |
| 5 | 36 |
| 6 | 14 |
| 7 |    |
| 8 |    |
| 9 |    |

|   |    |
|---|----|
| 1 | 25 |
| 2 | 12 |
| 3 | 47 |
| 4 | 89 |
| 5 | 36 |
| 6 | 14 |
| 7 | 99 |
| 8 |    |
| 9 |    |

99插入

|   |    |
|---|----|
| 1 | 25 |
| 2 | 12 |
| 3 | 47 |
| 4 | 89 |
| 5 | 36 |
| 6 | 14 |
| 7 |    |
| 8 |    |
| 9 |    |

99插入

|   |    |
|---|----|
| 1 | 25 |
| 2 | 12 |
| 3 | 47 |
| 4 | 89 |
| 5 | 36 |
| 6 | 14 |
| 7 |    |
| 8 |    |
| 9 |    |

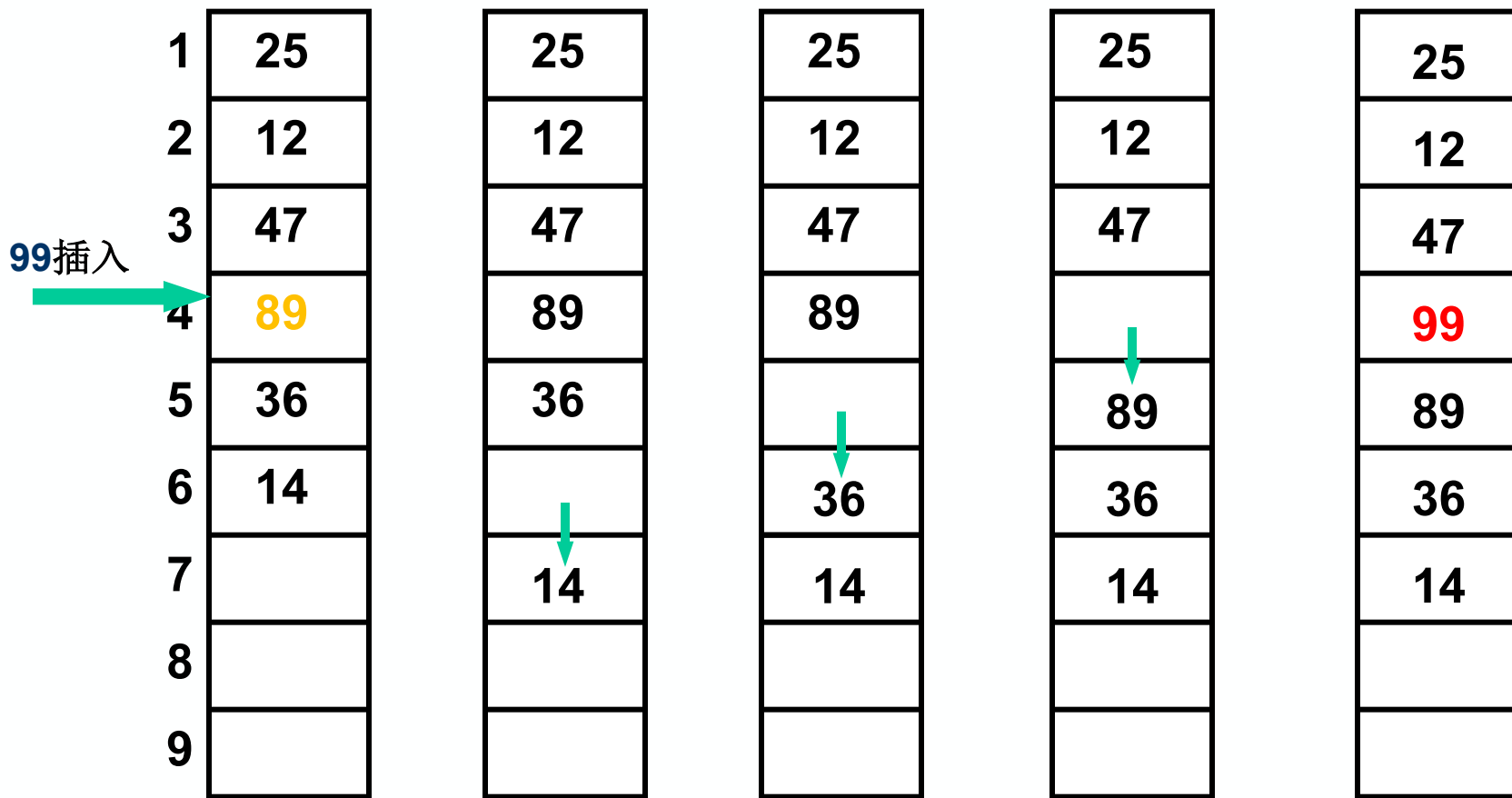
①插入位置在表尾

此时地址连续, 无任何空出单元

②插入位置在表中

③插入位置在表首

## 4. 插入 (在顺序表L中第i个位置插入数据元素e)



最麻烦的是插入在最前面，当 $i=1$ 时， $n$ 个元素都需要后移挪位并且只能从最后一个元素开始后移，否则会覆盖其他元素的值。

## 4. 插入 (在顺序表L中第*i*个位置插入数据元素*e*)

对于本算法来说，元素移动的次数不仅与表长  $L.length=n$  有关，而且与插入位置第  $i$  个有关。

- 当  $i=n+1$  时，移动次数为 0；

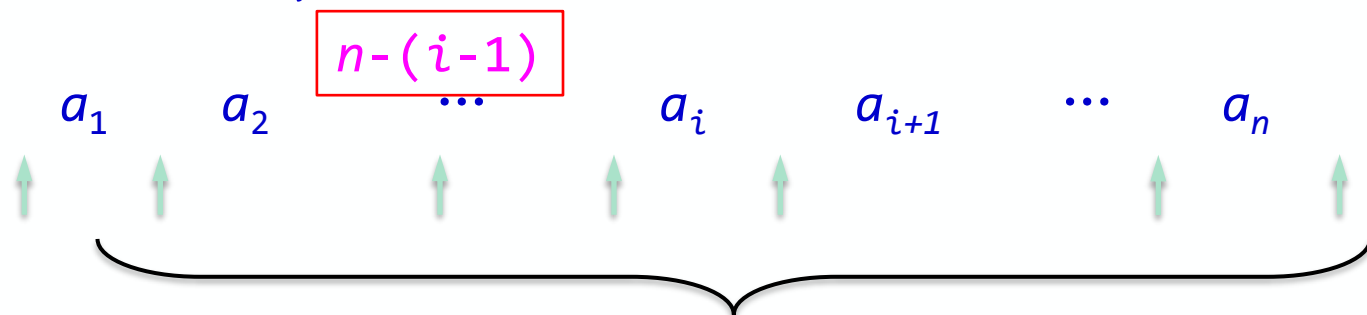
- 当  $i=1$  时，移动次数为  $n$ ，达到最大值。

算法最好时间复杂度为  $O(1)$

算法最坏时间复杂度为  $O(n)$

## 4. 插入 (在顺序表L中第i个位置插入数据元素e)

- 平均情况分析，先分析可插入位置：



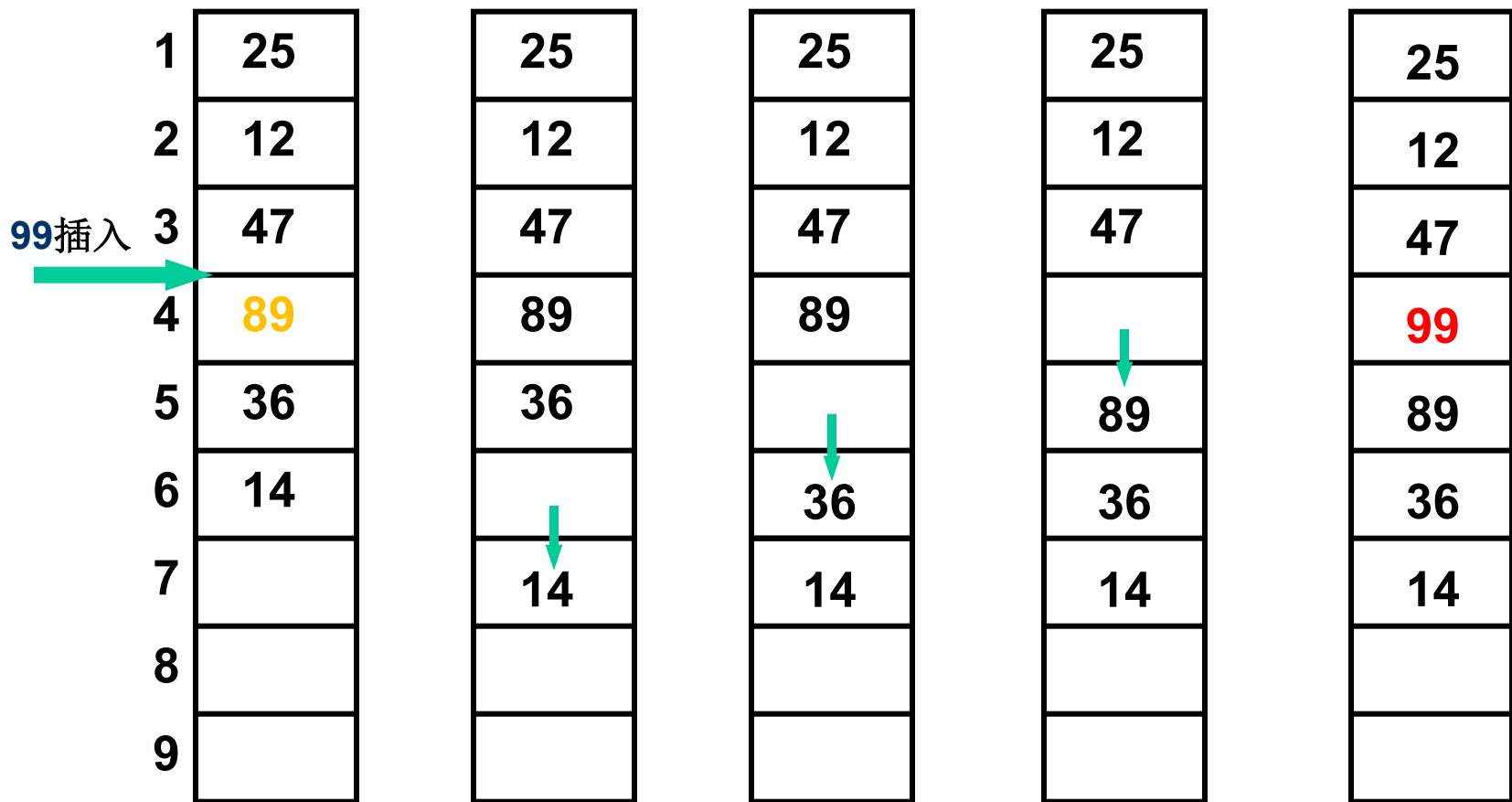
在线性表L中共有  $n+1$  个可以插入元素的地方

- 在第  $i$  个位置插入，若为等概率情况，则  $p_i = \frac{1}{n+1}$
- 此时需要将  $a_i \sim a_n$  的元素均后移一个位置，共移动  $n-i+1$  个元素。
- 在长度为  $n$  的线性表中插入一个元素时所需移动元素的平均次数为：

$$\sum_{i=1}^{n+1} p_i (n-i+1) = \sum_{i=1}^{n+1} \frac{1}{n+1} (n-i+1) = \frac{n}{2}$$

因此插入算法的平均  
时间复杂度为：  $O(n)$

## 4. 插入 (在顺序表L中第i个位置插入数据元素e)



```
for(j=L.length-1; j>=i-1; j--) //从第n个元素到第i个元素
    L.elem[j+1]=L.elem[j]; //插入位置及之后的元素后移
L.elem[i-1]=e; //将新元素e放入第i个位置
```

## 4. 插入 (在顺序表L中第i个位置插入数据元素e) P28

在表的第 $i$  ( $1 \leq i \leq n+1$ ) 个位置上插入一个新结点 $e$ , 使长度为 $n$ 的线性表( $a_1, \dots, a_{i-1}, a_i, \dots, a_n$ ) 变成长度为 $n+1$ 的线性表( $a_1, \dots, a_{i-1}, e, a_i, \dots, a_n$ )。

Status ListInsert\_Sq(SqList &L, int i, ElemType e)

{

if( $i < 1$  ||  $i > L.length + 1$ ) //  $i$  值不合法

return ERROR;

if( $L.length == MAXSIZE$ )

return ERROR; // 当前存储空间已满

for( $j = L.length - 1; j \geq i - 1; j--$ ) // 循环从最后一个元素开始移动

$L.elem[j+1] = L.elem[j];$  // 插入位置及之后的元素后移

$L.elem[i-1] = e;$  // 将新元素 $e$ 放入第 $i$ 个位置

$L.length++;$  // 表长增1

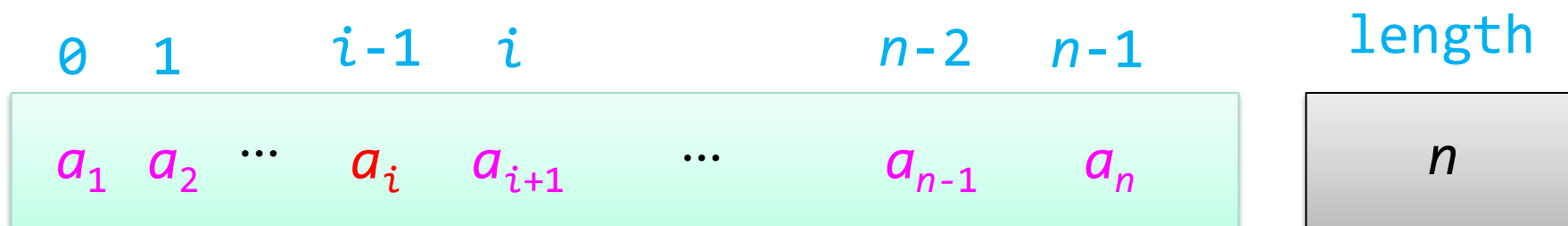
return OK;

}



## 5. 删除 (将顺序表L中第i个数据元素删除) P28

该运算删除顺序表L的第 $i$  ( $1 \leq i \leq L.Length$ ) 个元素。



e

删除完成!

- ①将欲删除的元素值保留在e中。
- ②将 $a_{i+1}$ 至 $a_n$ 依次向前移动一个位置，中间不能有任何空出的存储单元。
- ③表长减1。

## 5. 删除 (将顺序表L中第*i*个数据元素删除)

对于删除算法来说，元素移动的次数也与表长  $n$  和删除元素的位置  $i$  有关。

- 当  $i=n$  时，移动次数为  $0$ ;

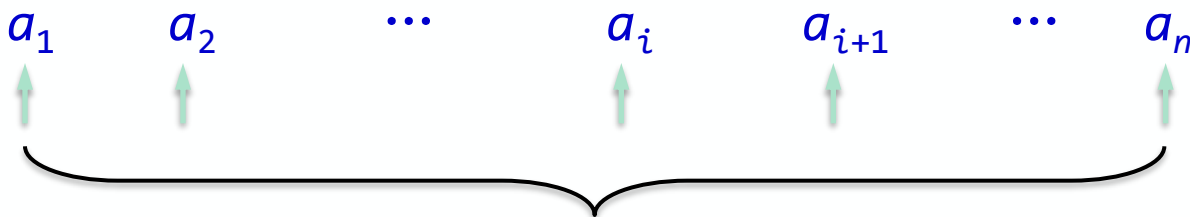
- 当  $i=1$  时，移动次数为  $n-1$ ，达到最大值。

删除算法最好时间复杂度为  $O(1)$

删除算法最坏时间复杂度为  $O(n)$

## 5. 删除 (将顺序表L中第i个数据元素删除) P29

### ■ 平均情况分析:



在线性表L中共有 $n$ 个可以删除元素的地方

■ 在删除元素 $a_i$ 时, 若为等概率情况, 则 $p_i = \frac{1}{n}$

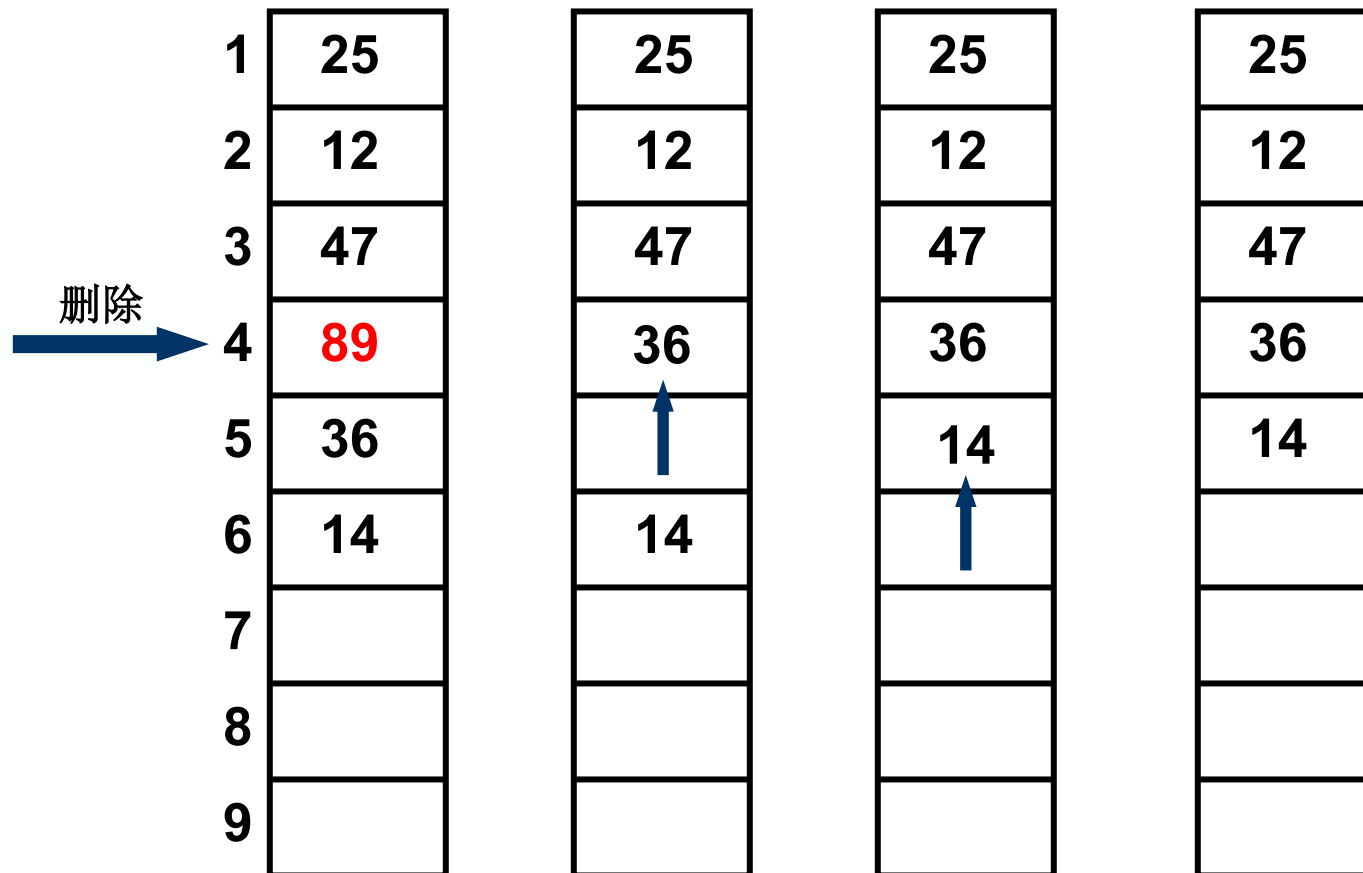
■ 需要将 $a_{i+1} \sim a_n$ 均前移一个位置, 共移动 $n-i$ 个元素。

■ 在长度为 $n$ 的线性表中删除一个元素时所需移动元素的平均次数为:

$$\sum_{i=1}^n p_i(n-i) = \sum_{i=1}^n \frac{1}{n}(n-i) = \frac{n-1}{2}$$

因此删除算法的平均  
时间复杂度为:  $O(n)$

## 5. 删除 (将顺序表L中第i个数据元素删除)



从第i+1个元素到最后一个元素，依次前移补空。

```
for (j=i;j<=L.length-1;j++)  
    L.elem[j-1]=L.elem[j];
```

## 5. 删除 (将顺序表L中第i个数据元素删除) P29

将表的第  $i$  ( $1 \leq i \leq n$ ) 个结点删除,  
使长度为  $n$  的线性表  $(a_1, \dots, a_{i-1}, a_i, a_{i+1}, \dots, a_n)$   
变成长度为  $n-1$  的线性表  $(a_1, \dots, a_{i-1}, a_{i+1}, \dots, a_n)$

Status ListDelete\_Sq(SqList &L, int i, int &e)

```
{
    if((i<1) || (i>L.length))
        return ERROR; //i值不合法
    e=L.elem[i-1];    //将被删除的第i个元素值赋给e
    for(j=i;j<=L.length-1;j++) //第i+1个元素至第n个元素
        L.elem[j-1]=L.elem[j]; //被删除元素之后的元素前移
    L.length--;        //表长减1
    return OK;         //删除成功返回OK
}
```

|       |       |       |       |       |       |  |
|-------|-------|-------|-------|-------|-------|--|
| 0     | 1     | 2     | 3     | 4     | 5     |  |
| $a_1$ | $a_2$ | $a_i$ | $a_4$ | $a_5$ | $a_6$ |  |

在一个长度为 $n$ 的顺序表中删除第 $i$ 个元素 ( $1 \leq i \leq n$ ) 时, 需向前移动 ( ) 个元素。

A.  $n$

B.  $i-1$

C.  $n-i$

D.  $n-i+1$

答案: C, 只需要前移第 $i+1$ 个到第 $n$ 个元素, 因此原本 $n$ 个元素, 有 $i$ 个元素不需要移动

**【多选】在n个元素的线性表的数组表示中，以下时间复杂度为 $O(1)$ 的操作是（ ）。**

- A. 访问第i个结点 ( $1 \leq i \leq n$ ) 和求第i个结点的直接前驱 ( $2 \leq i \leq n$ )**
- B. 在最后一个结点后插入一个新的结点**
- C. 删除第1个结点**
- D. 在第i个结点 ( $1 \leq i \leq n$ ) 后插入一个结点**

**答案：AB，A是随机存取特性，B中，在最后位置插入新结点不需要移动元素，时间复杂度为 $O(1)$ ；C被删结点之后的结点需依次前移，时间复杂度为 $O(n)$ ；D需要后移 $n-i$ 个结点，时间复杂度为 $O(n)$ 。**

# 顺序表中基本操作的实现

| 操作    | 平均时间复杂度 |
|-------|---------|
| 按序号取值 | $O(1)$  |
| 按值查找  | $O(n)$  |
| 插入    | $O(n)$  |
| 删除    | $O(n)$  |

# 顺序表（顺序存储结构）的特点 P29

- (1) 利用数据元素的存储位置表示线性表中相邻数据元素之间的前后关系，即**逻辑结构与存储结构一致**。
- (2) **随机存取法**：随机访问，通过首地址和元素序号可以在 $O(1)$ 时间内快速地计算出任何一个数据元素的存储地址。

## 缺点：

- (1) 在插入、删除表中某一元素时，需要移动大量元素。
  - (2) 属于静态存储形式，数据元素的个数不能自由扩充(固定长度)。
- 链表中数据元素的个数可以自由扩充

## 2.5 线性表的链式表示和实现 P29

---

- 用一组物理位置任意的存储单元来存放线性表的数据元素。
- 这组存储单元既可以是连续的，也可以是不连续的，甚至是零散分布在内存中的任意位置上的。
- 链式存储结构不要求逻辑上相邻的元素在物理位置上也相邻，因此链表中元素的逻辑次序和物理次序不一定相同。

# 2.5 线性表的链式表示和实现 (P30)

第一个 第二个 第三个

例：线性表 (赵, 钱, 孙, 李, 周, 吴, 郑, 王)

顺序表

| 存储地址 | 存储状态 |
|------|------|
| 31   | 赵    |
| 33   | 钱    |
| 35   | 孙    |
| 37   | 李    |
| 39   | 周    |
| 41   | 吴    |
| 43   | 郑    |
| 45   | 王    |

链表

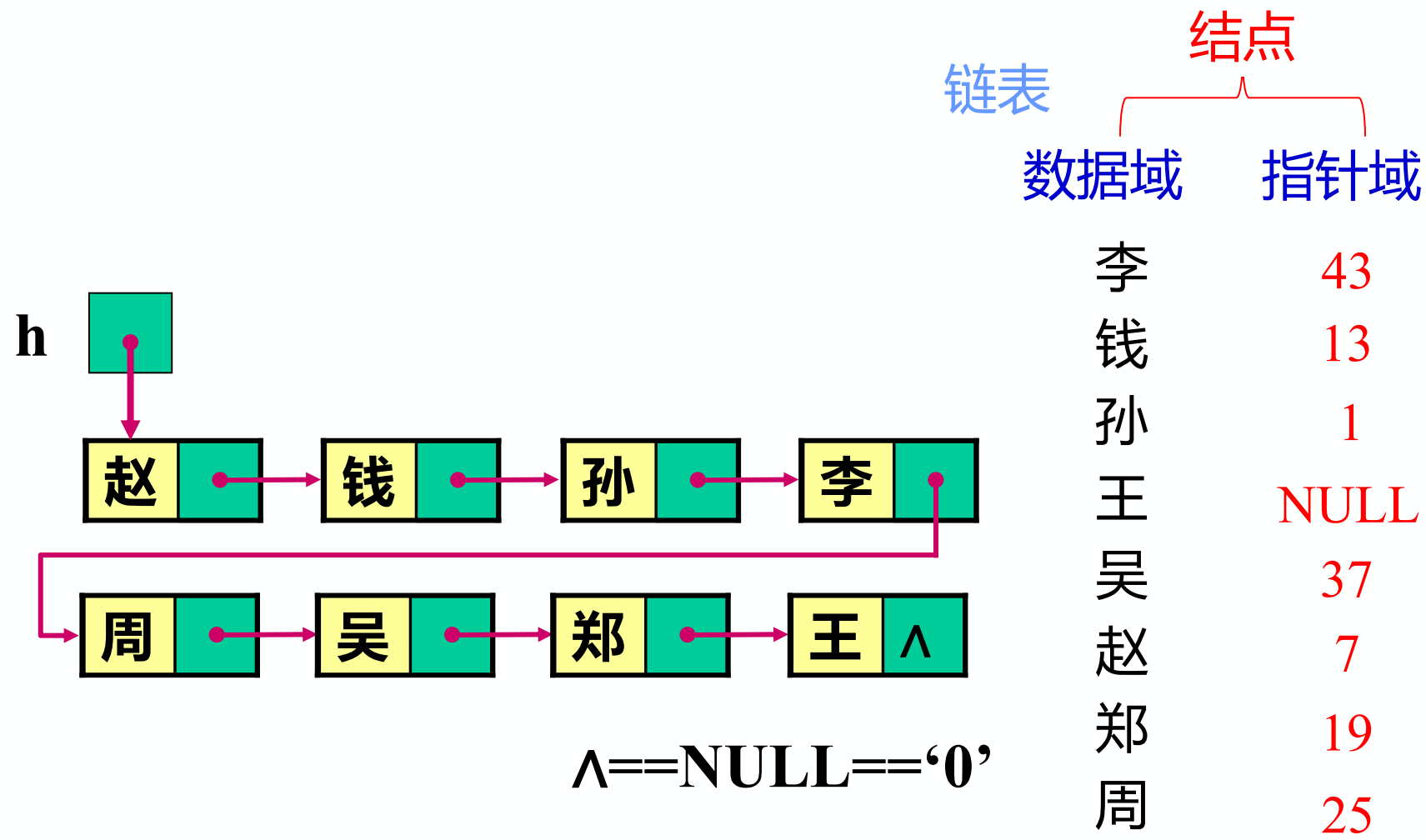
结点

| 存储地址 | 数据域 | 指针域  |
|------|-----|------|
| 1    | 李   | 43   |
| 7    | 钱   | 13   |
| 13   | 孙   | 1    |
| 19   | 王   | NULL |
| 25   | 吴   | 37   |
| 31   | 赵   | 7    |
| 37   | 郑   | 19   |
| 43   | 周   | 25   |

# 2.5 线性表的链式表示和实现 (P30)

第一个 第二个 第三个

例：线性表 (赵, 钱, 孙, 李, 周, 吴, 郑, 王)



# 与链式存储有关的术语 (P30&31)

1、**结点**：链表中数据元素的存储映像。由数据域和指针域两部分组成：



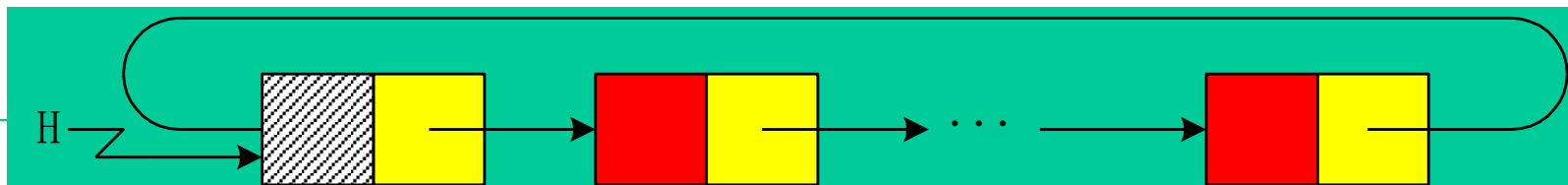
2、**链表**：n个结点由**指针链**组成一个链表。

3、**单链表**：链表的每个结点中只包含一个指针域。

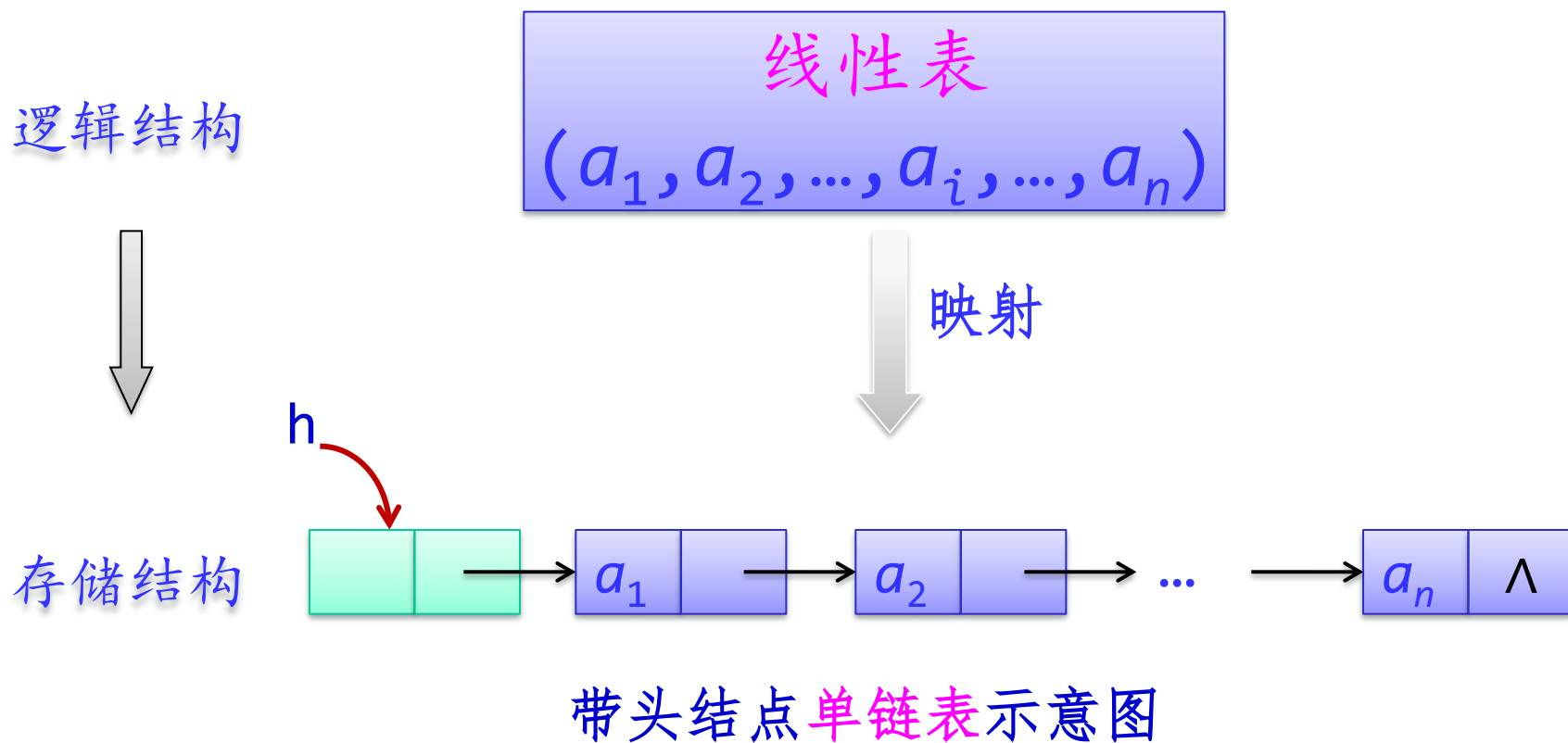
4、**双链表**：链表的每个结点中包含两个指针域。



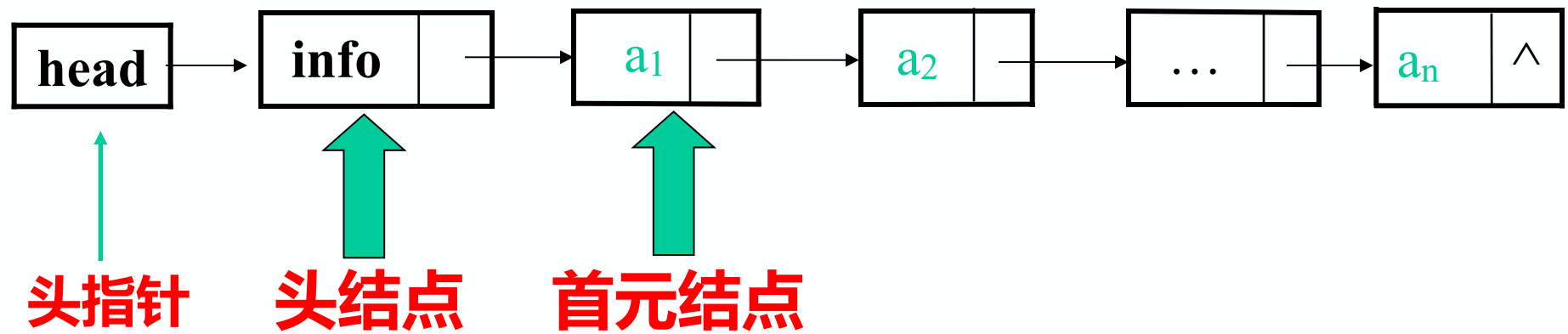
5、**循环链表**：尾结点的指针域指向头结点，形成环。



# 线性表的链式表示和实现



# 头指针、头结点和首元结点(P30)



**头指针**：指向链表中第一个结点的指针

**首元结点**：链表中存储第一个数据元素 $a_1$ 的结点

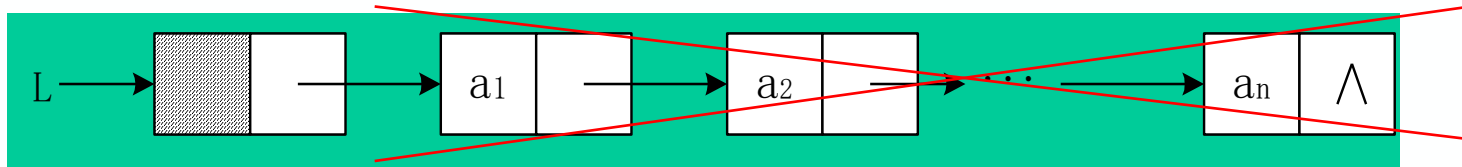
**头结点**：单链表为了处理方便，会在首元结点前附设一个结点，数据域内可存放表长等信息，但此结点不计入表长  
(一个单链表只包含一个头结点)

# 单链表的定义和实现(P32)

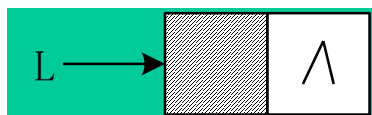


## ➤ 有头结点的单链表

非空表



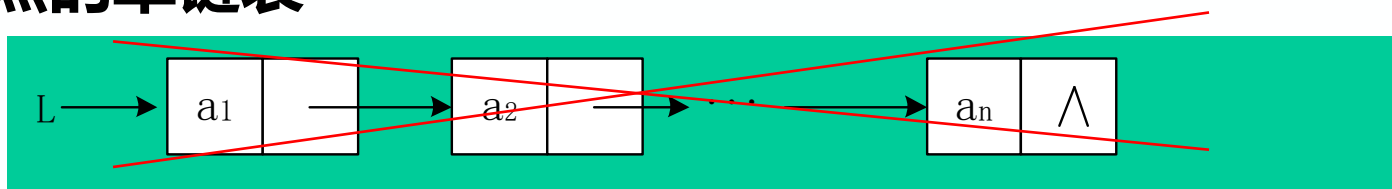
空表



头结点的指针域为空即表示空表

## ➤ 无头结点的单链表

非空表

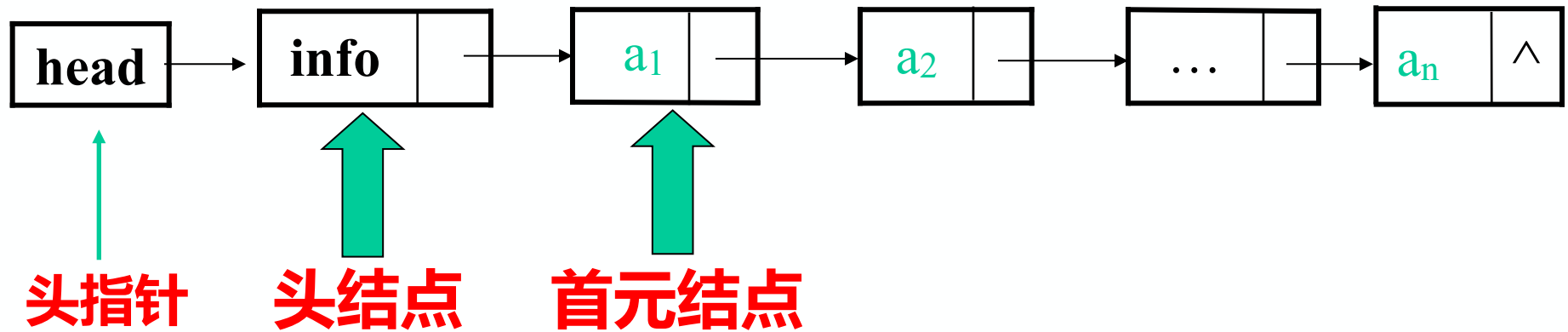


空表



头指针为空即表示空表

# 引入头结点的优点(P31)



- **便于首元结点的处理**：首元结点的地址保存在头结点的指针域中，故首元结点的前驱也是包含数据域和指针域的结点，与其他位置上结点无异。因此**首元结点的操作和表中其他结点的操作相一致**，无需进行特殊处理。
- **便于空表和非空表的统一处理**：无论链表是否为空，都有一个头结点，头指针都是指向头结点的非空指针，因此**空表和非空表的处理也就统一了**。

# 链表（链式存储结构）的特点

**(1)结点在存储器中的位置是任意的，即逻辑上相邻的数据元素在物理位置上不一定相邻。**

“顺藤摸瓜”

顺序存取

**(2)访问时只能通过头指针进入链表，并通过每个结点的指针域依次向后顺序扫描其余结点，所以寻找第一个结点和最后一个结点所花费的时间不等。**

顺序表（顺序存储结构）：随机存取

**链接存储的存储结构所占存储空间（ ）**

- A. 分两部分，一部分存放结点值，另一部分存放结点所占单元数**
- B. 只有一部分，存放结点值**
- C. 只有一部分，存储表示结点间关系的指针**
- D. 分两部分，一部分存放结点数据值，另一部分存放表示结点间关系的指针**

**答案：D**

将长度为 $n$ 的单链表链接在长度为 $m$ 的单链表后面，其算法的时间复杂度是（ ）。

A.  $O(1)$

B.  $O(n)$

C.  $O(m)$

D.  $O(m+n)$

将长度为 $m$ 的单链表链接在长度为 $n$ 的单链表后面： $O(n)$

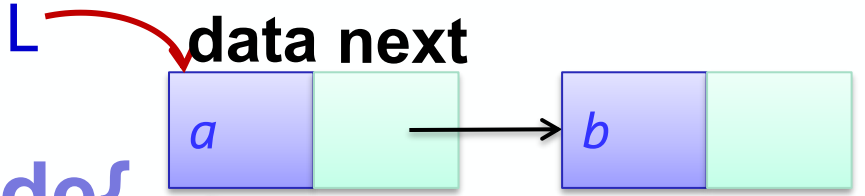
**答案：C**

解释：需要先遍历长度为 $m$ 的单链表，找到这个长度为 $m$ 的单链表的尾结点，然后将其指针域置为另一个单链表的首元结点的地址。

# 单链表中结点的类型定义 (P30)

类型定义:

```
typedef struct Lnode{  
    ElemType data;  
    struct Lnode *next;  
} Lnode, *LinkList; //普通的结点类型, 指针类型
```



//结点的数据域  
//结点的指针域

定义结点x: **Lnode x;**

x.data //结点x的数据域

x.next //结点x的指针域

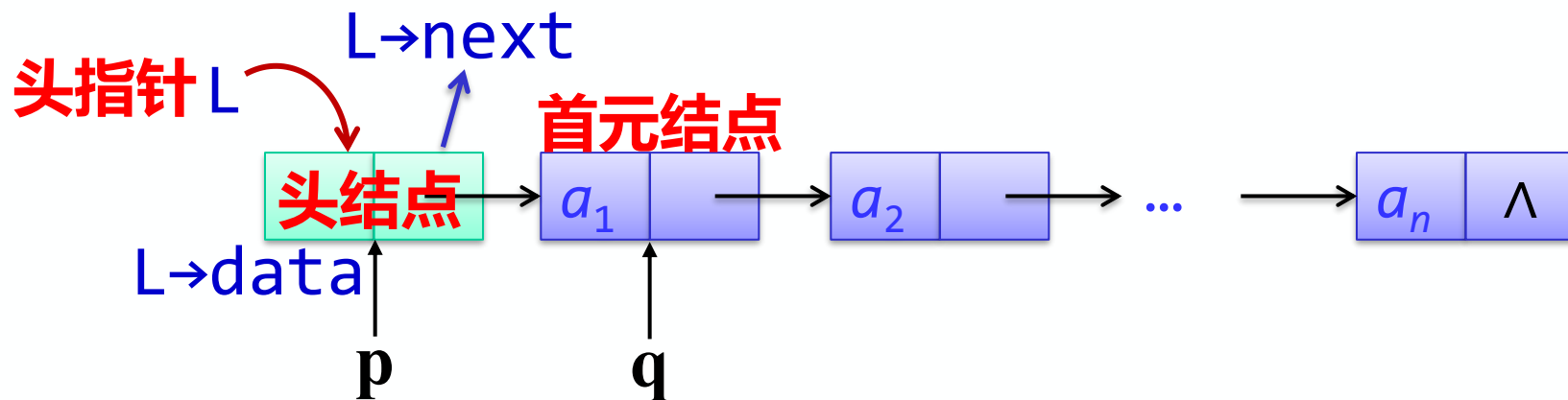
定义指向结点的指针L: **Lnode \*L;** ↔ **LinkList L;**

L→data

L→next

通常用 **LinkList L;** 来定义单链表, L是单链表的头指针, 该链表又称为表L。

# 单链表的重要操作



`LinkList L;` //定义单链表的头指针L

`L→next→data=a1;` //利用指针L表示元素 $a_1$

`p = L;` //指针p指向头结点，存放头结点的地址

`q = L→next;` //q指向首元结点

`p = p→next;` //p指针后移，指向下一结点

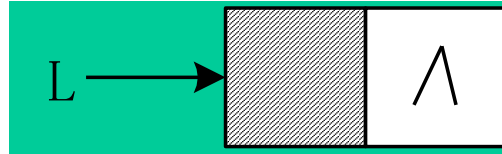
单链表的特点：当访问过一个结点后，只能接着访问它的后继结点，而无法访问它的前驱结点。

## 2.5.2 单链表基本操作的实现



1. 初始化
2. 按序号查找
3. 插入
4. 删除
5. 前插法建单链表
6. 后插法建单链表

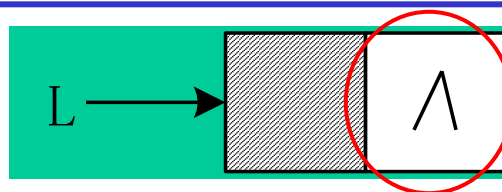
# 1. 初始化 (构造一个空的带头结点的单链表) P32



## 【算法描述】

```
Status InitList_L(LinkList &L)//通过引用参数L获得生成的新表
{
    L=new LNode;
    //生成新结点作头结点, 用头指针L指向头结点
    L→next=NULL;    //头结点的指针域置空
    return OK;
}
```

## 2. 判断链表是否为空



### 【算法描述】

```
int ListEmpty(LinkList L) //若L为空表返回1, 否则返回0
{
    if(L→next)    //非空  等同于 if(L→next!=NULL)
        return 0;
    else    //空
        return 1;
}
```

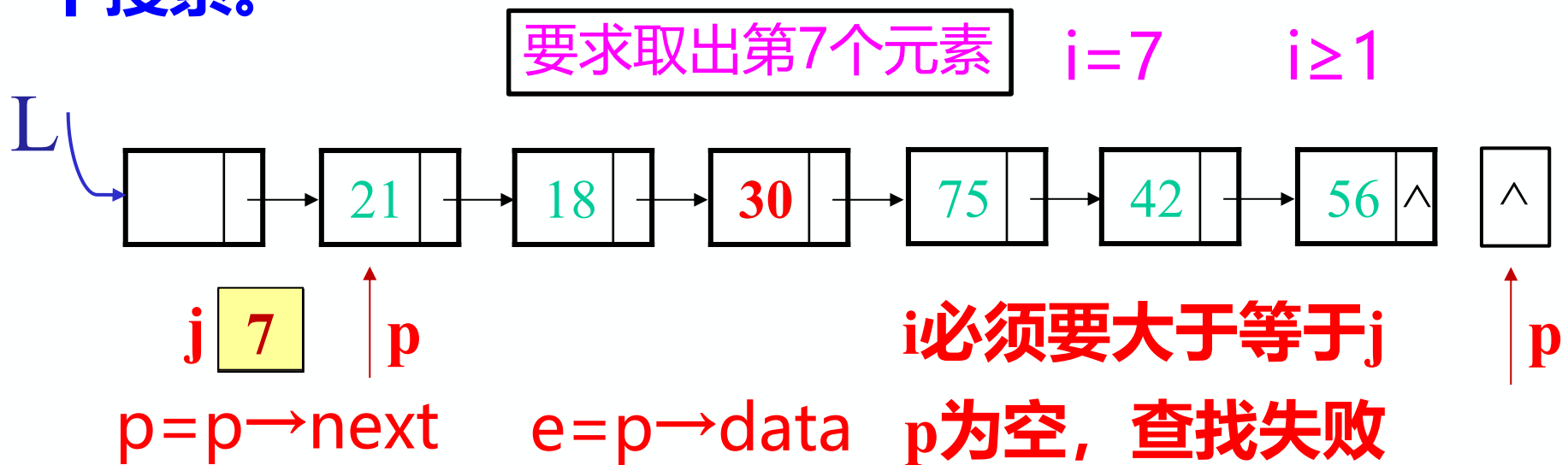
### 3. 按序号查找 (获取第*i*个位置上数据元素的内容) P32

- 思考：顺序表里如何找到第*i*个元素的数据元素内容？

**L.elem[i-1]**

#### ➤ 链表：

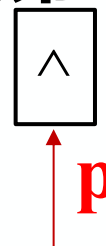
- (1) 逻辑相邻的结点不一定存储在物理相邻的单元中；
- (2) 根据给定的结点位置序号*i*，在链表中获取该结点的值不能像顺序表那样随机访问；
- (3) 链表只能从首元结点出发，**顺着指针域逐个结点向下搜索。**



### 3. 按序号查找 (根据位置*i*获取相应位置数据元素的内容)

#### 算法步骤: 【算法2.7 P32】

- (1)用指针**p**指向**首元结点**，从第一个元素结点开始搜索，**j**做计数器，累计当前扫描过的结点数，**j**初值为1；
- (2)从首元结点开始依次顺着next域向下访问，只要指针p不为空 (NULL)，且未到达第*i*个结点( $j < i$ )，则p不断右移指向下一个结点，计数器j相应加1；
- (3)退出循环时，若指针p为空（要找的元素位置超过链表长），或者计数器j大于i，说明序号i值不合法，查找失败；
- (4)否则取值成功，此时 $j = i$ ，p所指结点就是要找的第i个结点，用参数e保存当前结点的数据域，返回OK。



### 3. 按序号查找 (根据位置i获取相应位置数据元素的内容)

Status GetElem\_L(LinkList L, int i, Elemtype &e)

{

p=L→next; j=1; //先让p指向首元结点, j初始化

while(p && j<i) //当条件满足一直往后找

{ //指针p没有指向NULL, 且j还没到第i个位置

p=p→next;

++j; //j计数器+1

}//p为空 or j≥i退出循环, 当j>i, 第i个元素不合理

if( !p || j>i) return ERROR; //不合理的情况

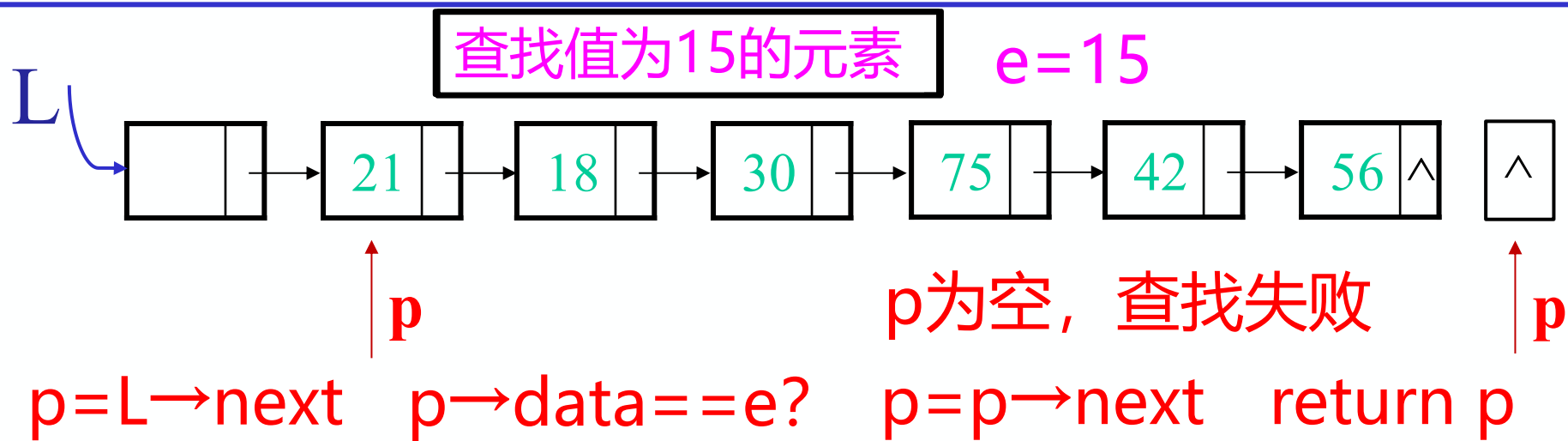
e=p→data; //j==i, 取第i个元素的值, 通过e带回

return OK;

时间复杂度:  $O(n)$

}

## 4. 按值查找 (查找给定值e的地址) P32

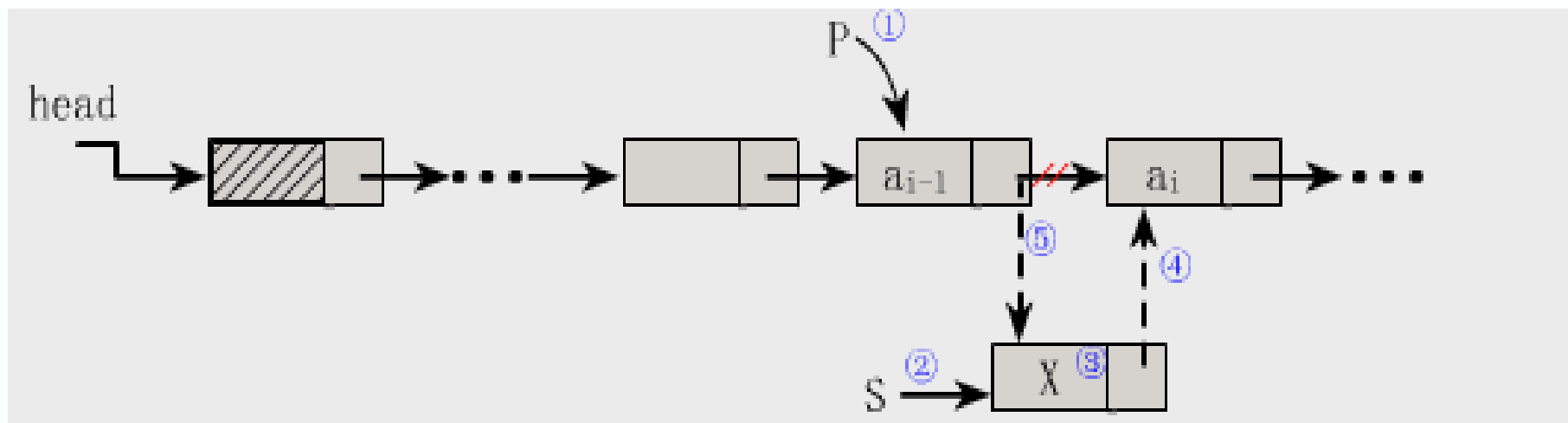
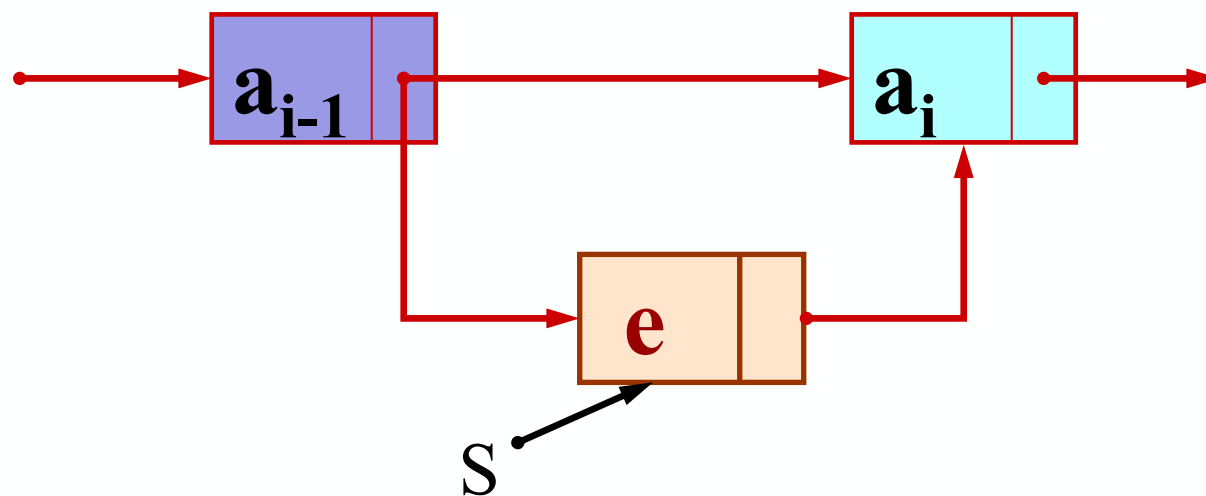


**LNNode \*LocateElem(LinkList L, Elemtype e)**

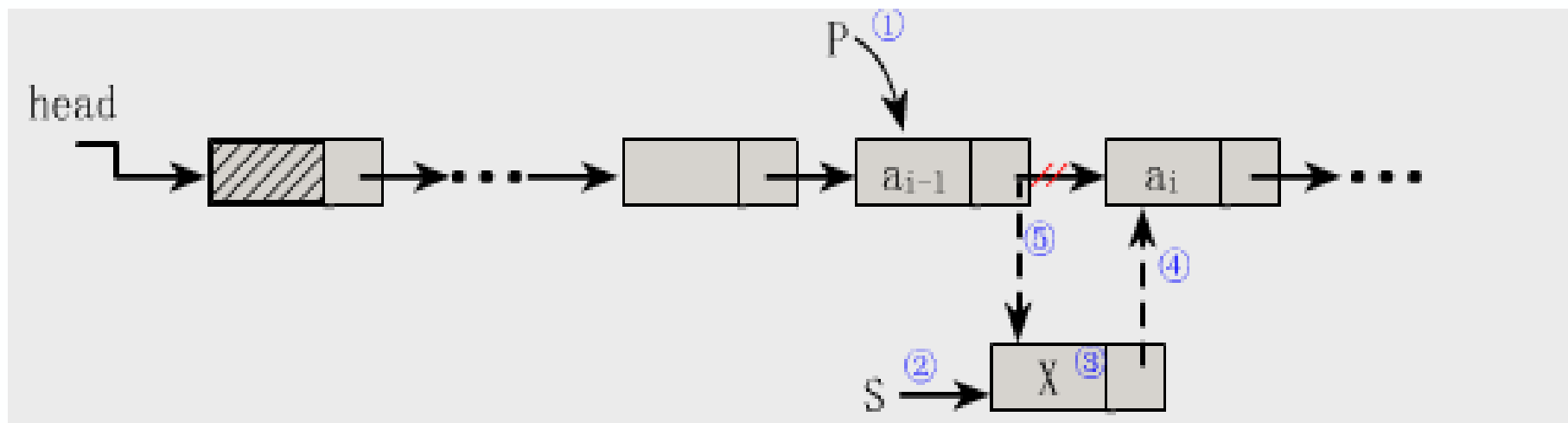
```
{ //在线性表L中查找值为e的数据元素
  //找到返回L中值为e的数据元素地址, 查找失败返回NULL
  p=L→next; //先让p指向首元结点
  while(p && p→data!=e) //当条件满足一直往后找
    //指针p没有指向NULL, 且p所指结点数据域不为e
    p=p→next;
  return p;
}
```

**时间复杂度:  $O(n)$**

# 5.单链表的插入(将新结点插入到表中第i个位置)



## 5.单链表的插入(将新结点插入到表中第i个位置)



步骤①先找到 $a_{i-1}$ 的存储位置，并由指针 $P$ 指向该结点

②生成一个新结点 $S$ ；③设置新结点的数据域为 $X$

插入新结点：④新结点的指针域指向结点 $a_i$

⑤结点 $a_{i-1}$ 的指针域指向新结点

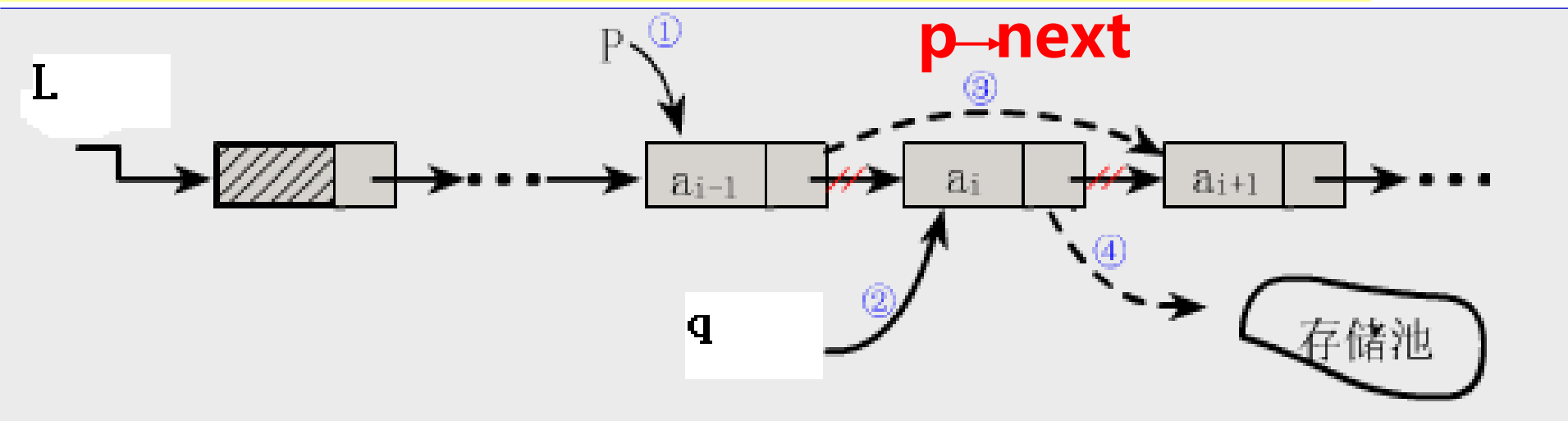
④ $S \rightarrow \text{next} = P \rightarrow \text{next}$ ;      ⑤ $P \rightarrow \text{next} = S$ ;

④⑤的顺序可以换吗？

## 5.单链表的插入(将新结点插入到表中第i个位置)

```
Status ListInsert_L( LinkList &L, int i, ElemType X){  
    p=L;  j=0; //可以插入到第一个位置  
    while(p && j<i-1) { p=p→next;    ++j; }  
    //寻找第i-1个结点, 令p指向该结点  
    if(!p || j>i-1)) return ERROR; //i大于表长+1或小于1, 不合理  
    s = new LNode; //生成新结点s  
    s→data = X; //将结点s的数据域置为X  
    s→next=p→next; //将结点s插入L中  
    p→next=s;  
    return OK;  
}
```

# 6. 单链表的删除 (删除第 i 个结点)



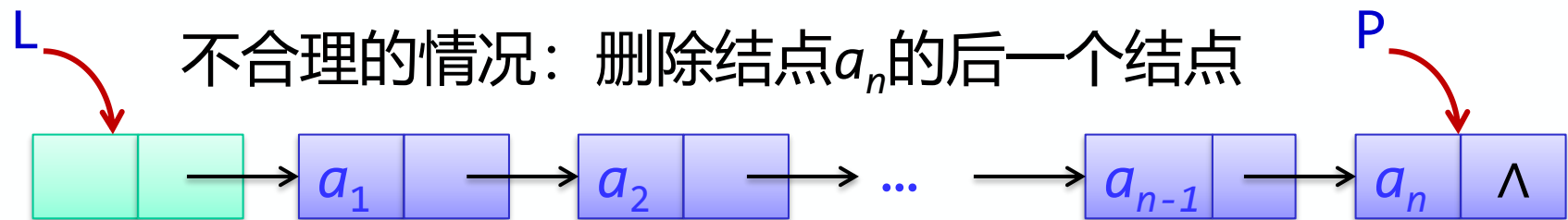
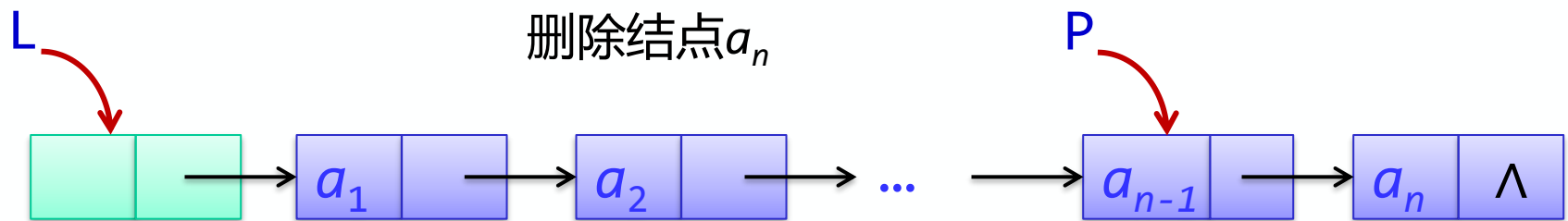
- 步骤 (1) 找到 $a_{i-1}$ 存储位置, 由指针 $p$ 指向
- (2) 临时保存待删除结点 $a_i$ 的地址在 $q$ 中, 以备释放
- (3) 结点 $a_{i-1}$ 的直接后继变为结点 $a_i$ 的后继结点 $a_{i+1}$
- (4) 释放结点 $a_i$ 的空间 free/delete

```
(3)①  $p \rightarrow next = q \rightarrow next;$   
(3)②  $p \rightarrow next = p \rightarrow next \rightarrow next;$ 
```

## 6. 删除 (单链表的第i个结点)

p 始终指向第i-1个结点。删除第i个结点, i的范围? **1-n**

若i的值不合理, 要么是 $j > i-1$  (j初值为0), 要么 $i \geq n+1$



此时 $p \rightarrow next = \text{NULL}$ , 表示不合理情况, 无法删除第 $n+1$ 个及之后不存在的结点

## 6. 删除 (单链表的第i个结点)

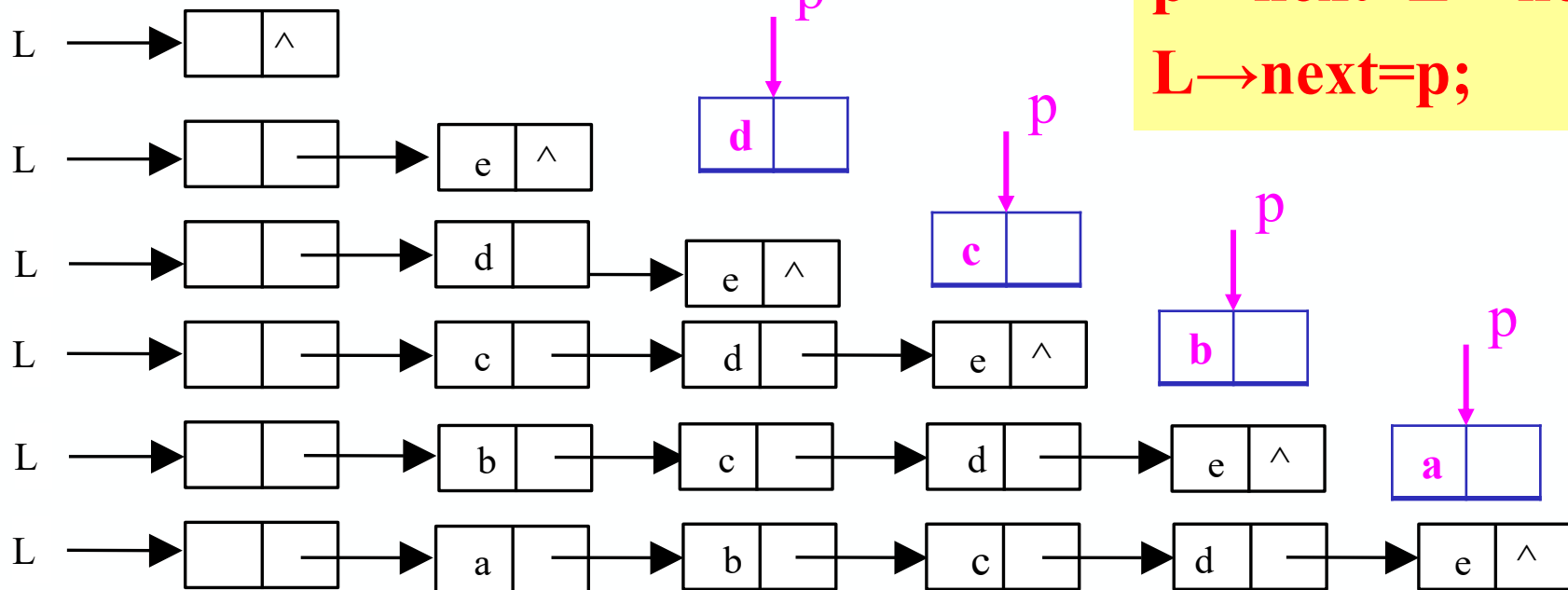
```
Status ListDelete_L( LinkList &L,int i,ElemType &e){  
    p=L; j=0;  
    while(p→next && j<i-1) { p=p→next; ++j; }  
    //寻找第i个结点, 令p指向其前驱 $a_{i-1}$ , i取值范围为1~n  
    if(!(p→next) || j>i-1)) return ERROR; //删除位置不合理  
    q=p→next; //临时保存被删结点的地址以备释放  
    p→next=q→next; //改变删除结点前驱结点的指针域  
    e=q→data; //将删除结点的数据域保存到引用参数e中  
    delete q; //释放删除结点的空间  
    return OK;  
}
```

【算法2.10】书中p35

## 7.单链表的建立 (头插法)

- 1、从一个空表开始，重复读入数据；
- 2、生成一个新结点，输入数据至新结点的数据域中
- 3、逆序从最后一个结点开始，逐个插入结点到链表的头部，即头结点之后

例：建立链表L (a,b,c,d,e)



```
L=new LNode;  
L->next=NULL;
```

```
p->next=L->next;  
L->next=p;
```

**注意：**链表中结点顺序与插入次序相反。

# 【算法2.11】 p36

```
void CreateList_F(LinkList
```

```
{
```

```
    L=new LNode;
```

```
    L->next=NULL;
```

```
    for( i=n;i>0;--i )
```

```
    {
```

```
        p=(LNode *)malloc(sizeof(LNode));
```

```
        p=new LNode;
```

```
        cin>>p->data;
```

```
        p->next=L->next;
```

```
        L->next=p;
```

```
    }
```

头插法建立单链表虽然简单，但生成的链表中结点的次序和输入数据的顺序不一致。希望两者次序一致，可采用尾插法

```
L=(LinkList)malloc(sizeof(LNode));
```

```
//先建立一个带头结点的单链表
```

```
//生成新结点
```

```
//输入元素值
```

```
scanf("%c",&p->data);
```

```
//插入到头结点之后
```

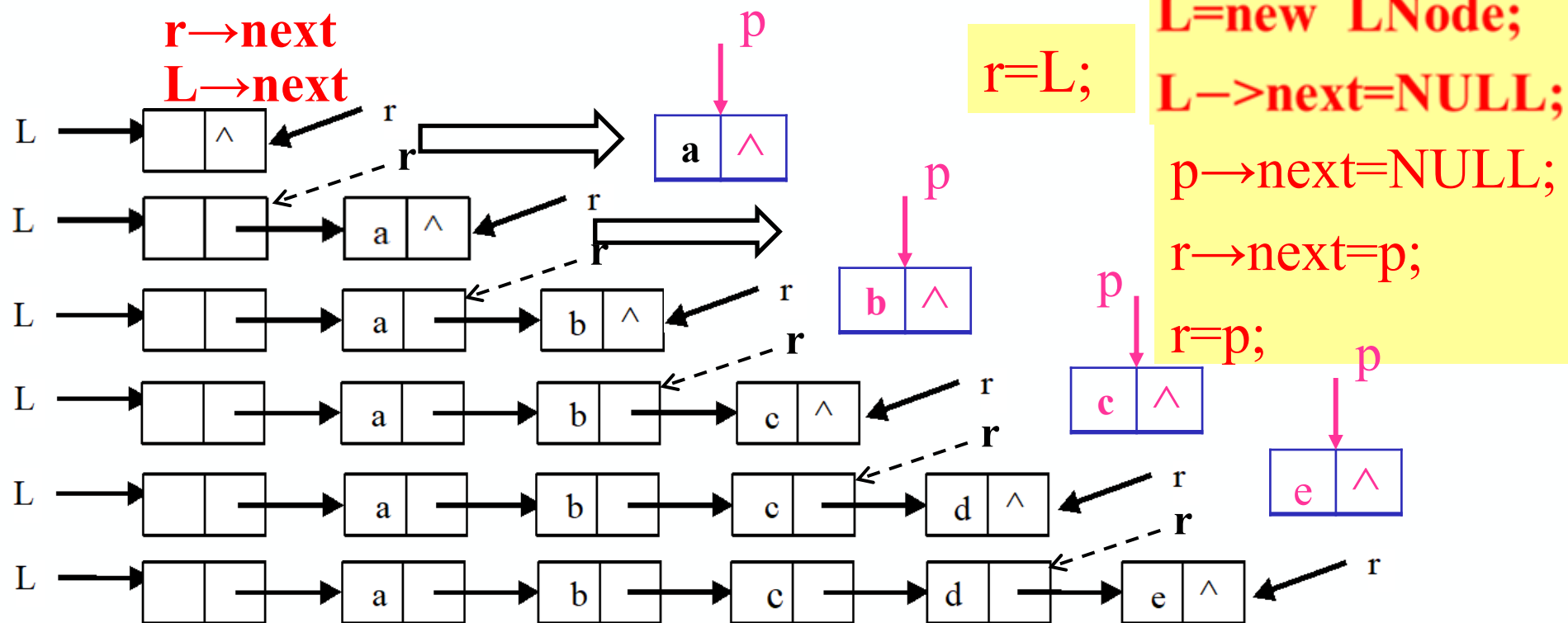
**时间复杂度：O(n)**

初始化

插入

## 8.单链表的建立 (尾插法)

- 1、从一个空表L开始，将新结点逐个插入到链表的尾部，尾指针r始终指向链表的尾结点。
- 2、初始时，r和L均指向头结点。每读入一个数据元素申请一个新结点，正序将新结点插入到尾结点后，r指向新结点



## 8.单链表的建立 (尾插法)

```
void CreateList_L(LinkList &L,int n)
```

```
{ L=new LNode;
```

```
  L->next=NULL;
```

```
  r=L;           //尾指针r指向头结点
```

```
  for(i=0;i<n;++i)
```

```
  {
```

```
    p=new LNode;           //生成新结点
```

```
    cin>>p->data;           //输入元素值
```

```
    p->next=NULL; r->next=p; //插入到表尾
```

```
    r=p;           //r指向新的尾结点
```

```
  }
```

```
}
```

**初始化**

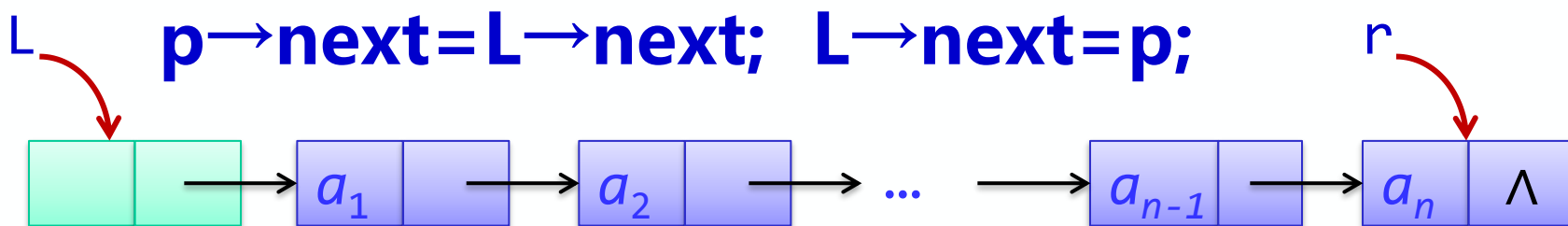
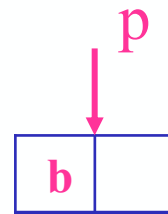
**插入**

**算法的时间复杂度:  $O(n)$**

$p \rightarrow next = NULL; r \rightarrow next = p; r = p;$

在一个长度为 $n$ 的带头结点的单链表 $L$ 上，设有尾指针 $r$ ，则执行（ ）操作与链表的表长有关。

- A. 在单链表首元结点前插入一个新结点
- B. 在单链表最后一个结点后插入一个新结点
- C. 删除单链表中的首元结点  $L \rightarrow next = L \rightarrow next \rightarrow next;$
- D. 删除单链表中最后一个结点



**答案：D。** A是头插法，B是尾插法，只需要插入一个结点，非 $n$ 个，因此只要修改指针，与元素个数 $n$ 无关。D：删除第 $n$ 个结点，就要先查找到第 $n-1$ 个结点，需要置结点 $a_n$ 的前驱 $a_{n-1}$ 指针域为 $NULL$ ，且需要让尾指针 $r$ 指向结点 $a_{n-1}$ 。

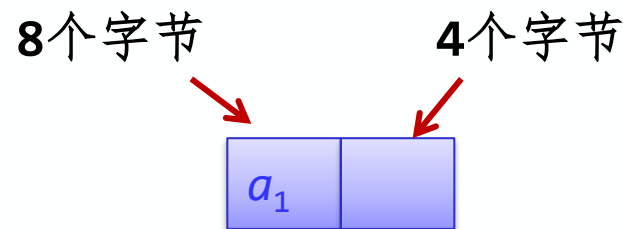
## 2.6 顺序表和链表的比较(P40)

- 1 空间性能的比较

- (1) 存储空间的分配

- 顺序表的存储空间必须预先分配，易造成存储空间浪费或空间溢出。
    - 链表的元素个数没有限制
    - 总结：当线性表的长度变化较大，难以预估存储规模时，宜采用链表作为存储结构。

## (2) 存储密度



$$\text{存储密度} = 8/12 = 0.67$$

• 存储密度 =  $\frac{\text{数据元素本身占用的存储量}}{\text{结点结构总共占用的存储量}}$

| 顺序表                                     | 链表                                |
|-----------------------------------------|-----------------------------------|
| 不用为表示结点间的逻辑关系而增加额外的存储开销， <b>存储密度等于1</b> | 需要借助指针来体现元素间的逻辑关系， <b>存储密度小于1</b> |

➤ 总结：当线性表的长度变化不大，为了提高存储效率、节约存储空间，宜采用顺序表

## • 2 时间性能的比较

- 顺序表随机存取，**链表**不具备随机存取特性，只能从表头**顺序存取**元素；
- 总结：若线性表的主要操作**取值**较多，很少做插入和删除，宜用**顺序表**
- 对于频繁进行**插入和删除**操作的线性表，宜采用**链表**。

---

**单链表的存储密度（ ）。**

**A. 大于1**

**B. 等于1**

**C. 小于1**

**D. 不能确定**

**答案： C**

---

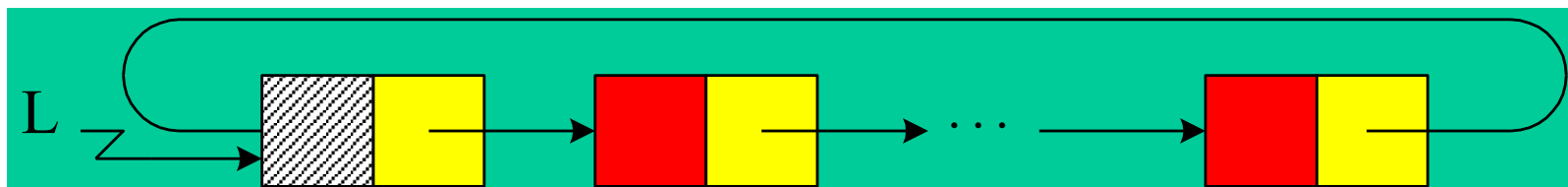
设线性表有 $2n$ 个元素，（ ）在单链表上实现要比顺序表上实现效率更高。

- A. 删除所有值为 $x$ 的元素
- B. 在最后一个元素的后面插入一个新元素
- C. 顺序输出前 $k$ 个元素
- D. 交换第 $i$ 个元素和第 $2n-i-1$ 个元素的值 ( $i=0, \dots, n-1$ )

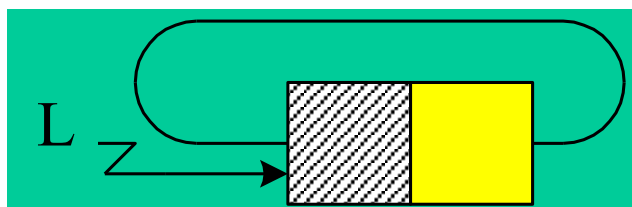
**答案：A**

解释：A中 比较并查找值为 $x$ 的元素，对于单链表和顺序表上实现的时间复杂度都为 $O(n)$ ，但后者要移动很多元素，所以在单链表上实现效率更高。B中单链表首先要顺着指针链查找到最后一个元素，顺序表 $O(1)$ 。C、D顺序表效率更高

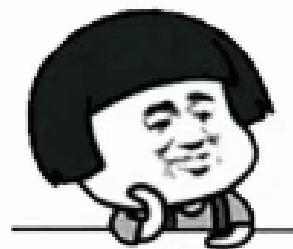
## 2.5.3 循环链表 (P38)



(a) 非空单循环链表

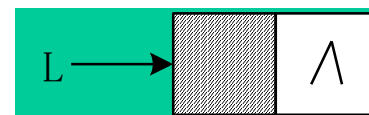


(b) 空的单循环链表



循环链表的  
判空条件?

空单链表



$L \rightarrow \text{next} == \text{NULL}$

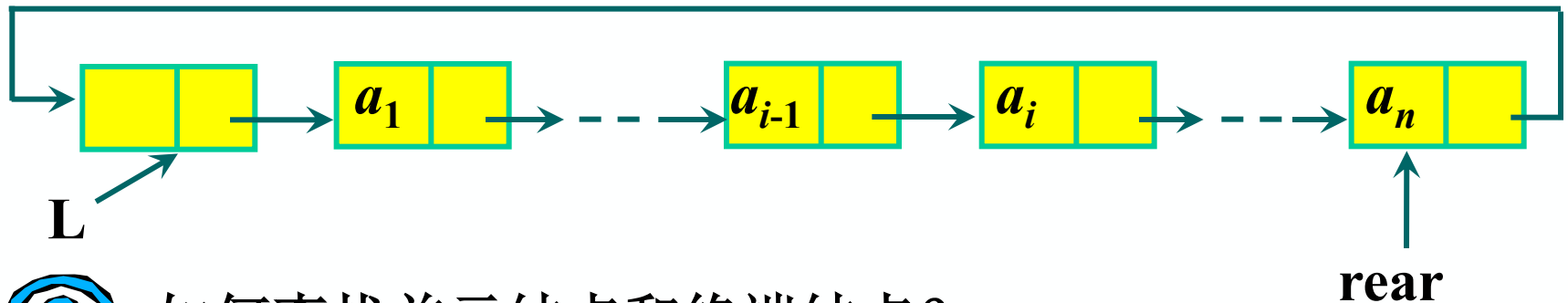
$L \rightarrow \text{next} == L$

循环链表的特点：最后一个结点的指针域指向头结点，整个链表形成一个环。（头尾相接）

循环链表中不存在NULL指针域，从表中任一结点出发均可找到表中其他结点。



对于循环链表，有时不给出头指针，而给出**尾指针**  
可以更方便地找到**首元结点**和**最后一个结点**



如何查找首元结点和终端结点？

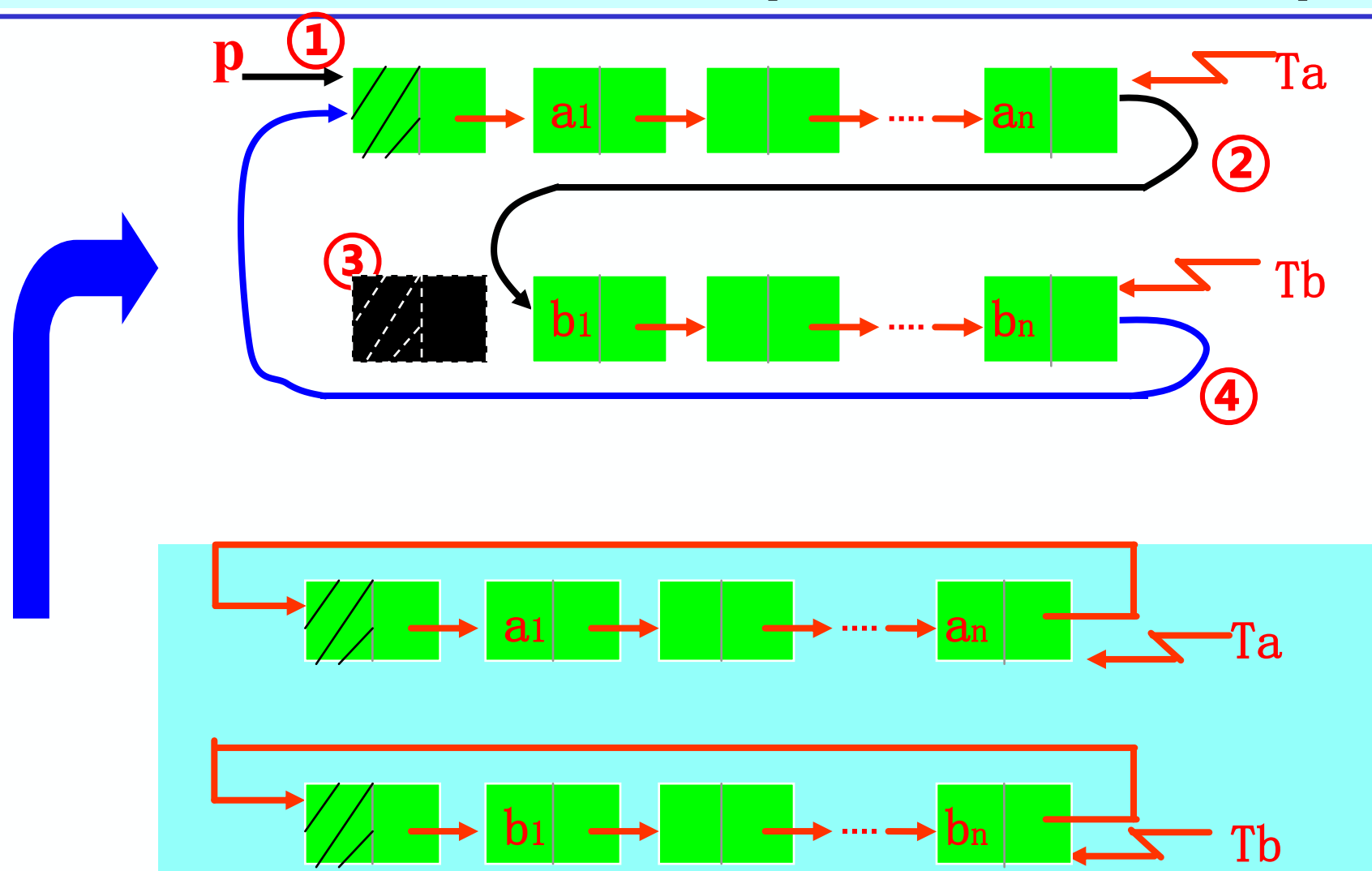
首元结点:  $\text{rear} \rightarrow \text{next} \rightarrow \text{next}$   
终端结点:  $\text{rear}$

时间复杂度:  $O(1)$

头指针表示  
单循环链表

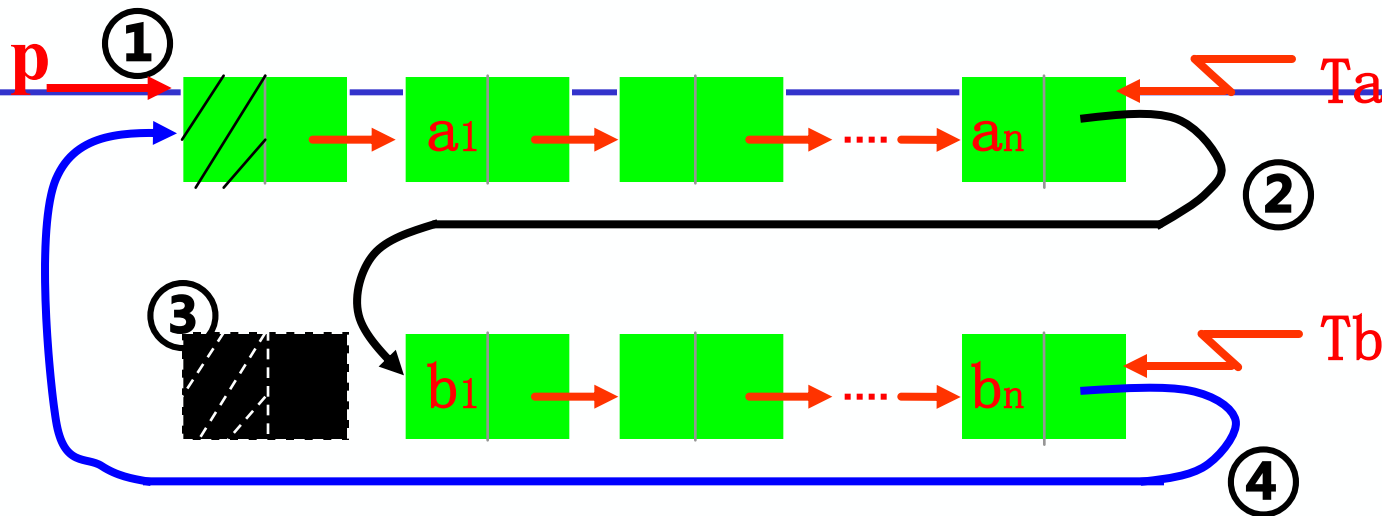
首元结点  $a_1$ :  $L \rightarrow \text{next}$  , 时间复杂度:  $O(1)$   
找尾结点  $a_n$  的时间复杂度:  $O(n)$

# 带尾指针的单循环链表合并（把Tb链接在Ta后）



- ① P存Ta头结点
- ② Tb首元结点链接Ta后面
- ③ 释放Tb头结点
- ④ 修改Tb指针的后继

# 单循环链表的合并



```
LinkList Connect(LinkList Ta, LinkList Tb)
```

```
{//假设Ta、Tb都是非空的单循环链表
```

```
    p = Ta → next;
```

```
//①p存Ta的头结点
```

```
    Ta → next = Tb → next → next;
```

```
//②Tb首元结点链接在Ta后面
```

```
    delete Tb → next;
```

```
//③释放Tb头结点
```

```
    Tb → next = p;
```

```
//④修改指针
```

```
    return Tb;
```

**$O(1)$**

```
}
```



时间复杂度  
是多少?

## 2.5.4 双向链表 (P38)



为什么要引入双向链表？

单链表、单循环链表的每个结点都指向其直接后继，

若要查找其直接前驱，需从头结点出发。

为解决其单向性，引入双向链表。

双向链表：

在单链表的基础上，增加了一个指向其直接前驱的指针域：prior

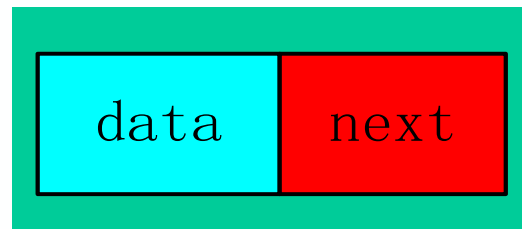
指向其直接后继的指针域：next



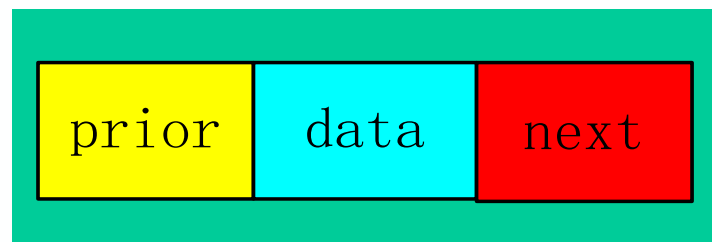
## 2.5.4 双向链表

### Double Linked Node

```
typedef struct DuLNode{  
    ElemType  data;  
    struct DuLNode *prior;  
    struct DuLNode *next;  
}DuLNode, *DuLinkList
```



单链表结点结构



双向链表的结点结构

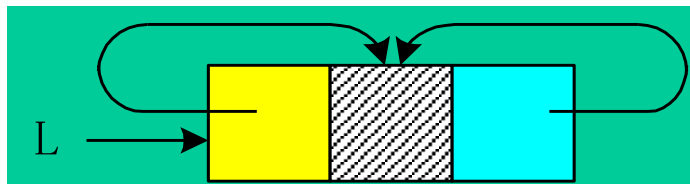
**双向链表的存储密度（ ）单链表的存储密度。**

- A. 大于**
- B. 等于**
- C. 小于**
- D. 不能确定**

**答案：C，分母更大**

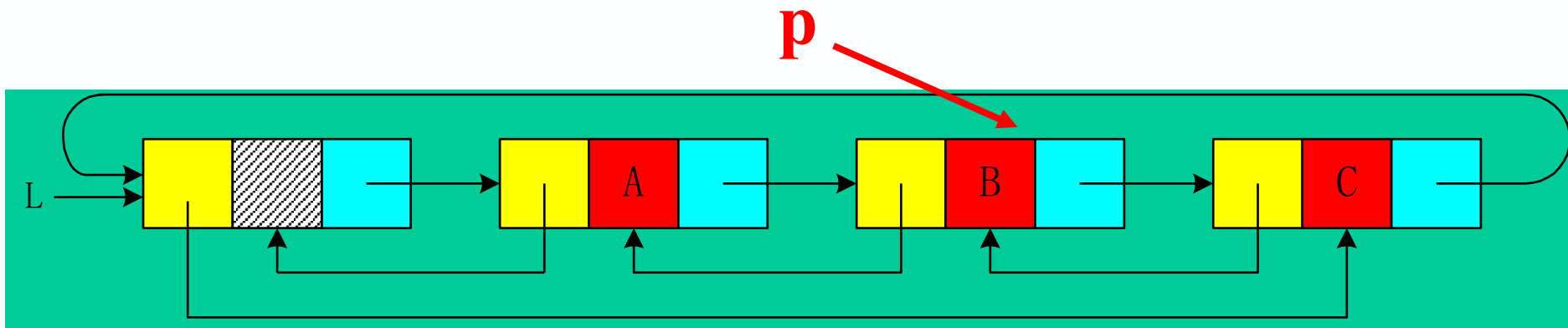
$$\text{存储密度} = \frac{\text{数据元素本身占用的存储量}}{\text{结点结构总共占用的存储量}}$$

# 双向循环链表 (P39)



$$L \rightarrow \text{prior} == L == L \rightarrow \text{next}$$

(a) 空的双向循环链表

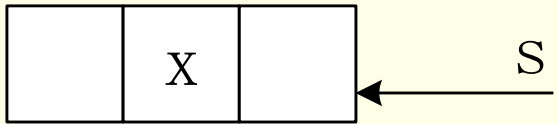
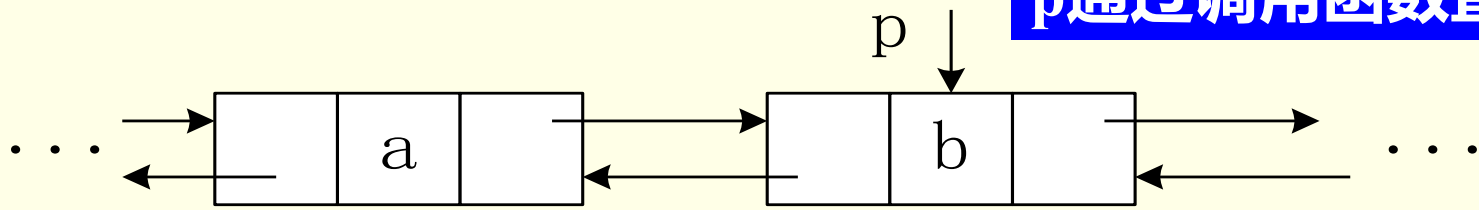


(b) 非空的双向循环链表 对称性

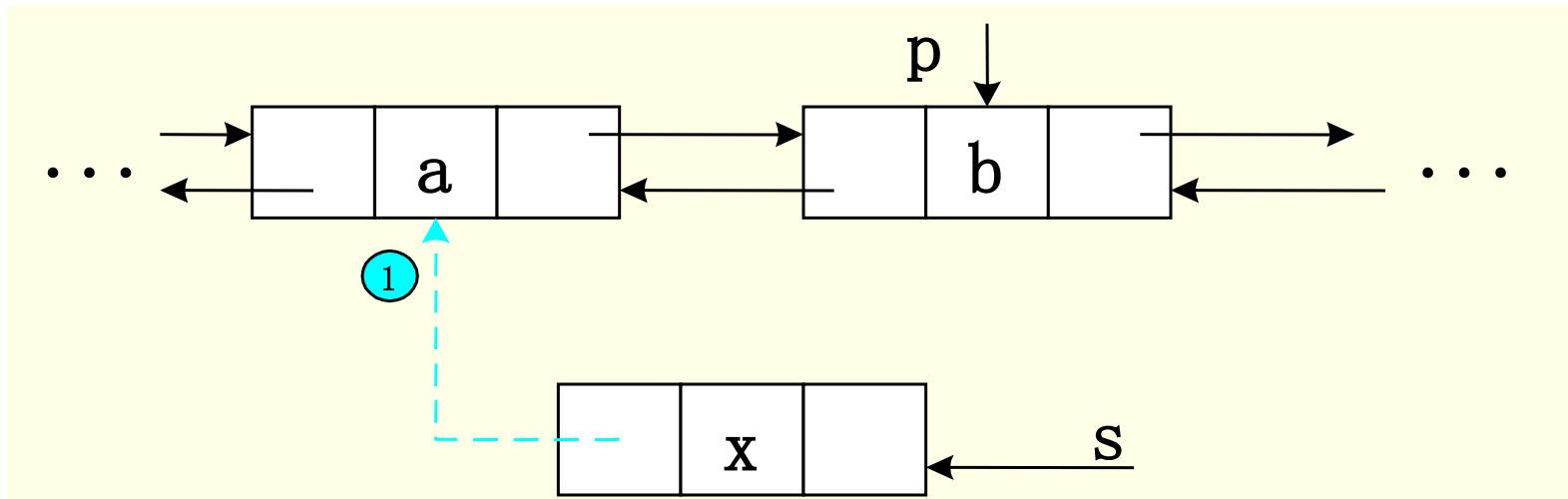
$$p \rightarrow \text{prior} \rightarrow \text{next} == p == p \rightarrow \text{next} \rightarrow \text{prior}$$

# 双向链表的插入【算法2.13】 P39

p通过调用函数直接得到

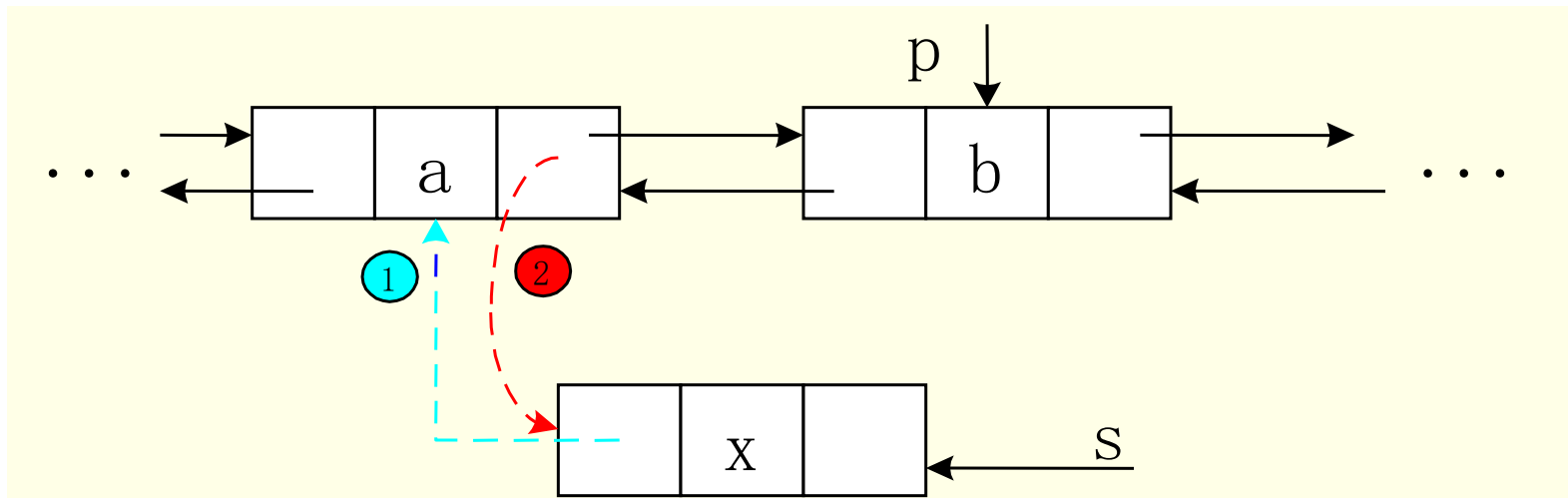


# 双向链表的插入【算法2.13】 P39



1.  $s \rightarrow \text{prior} = p \rightarrow \text{prior};$

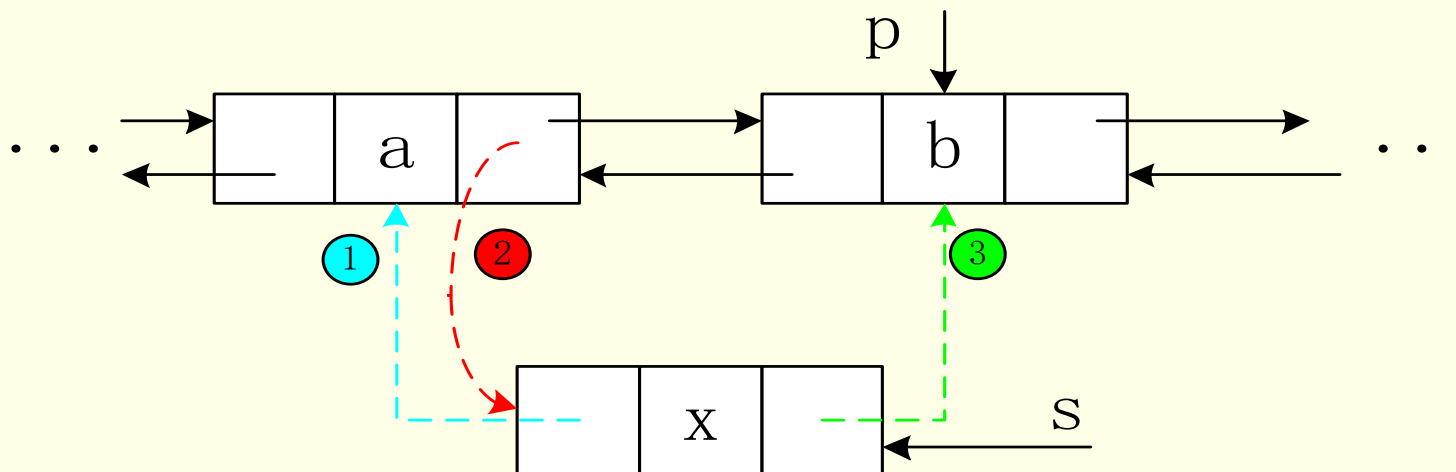
# 双向链表的插入【算法2.13】 P39



**1.  $s \rightarrow \text{prior} = p \rightarrow \text{prior};$**

**2.  $p \rightarrow \text{prior} \rightarrow \text{next} = s;$**

# 双向链表的插入【算法2.13】 P39

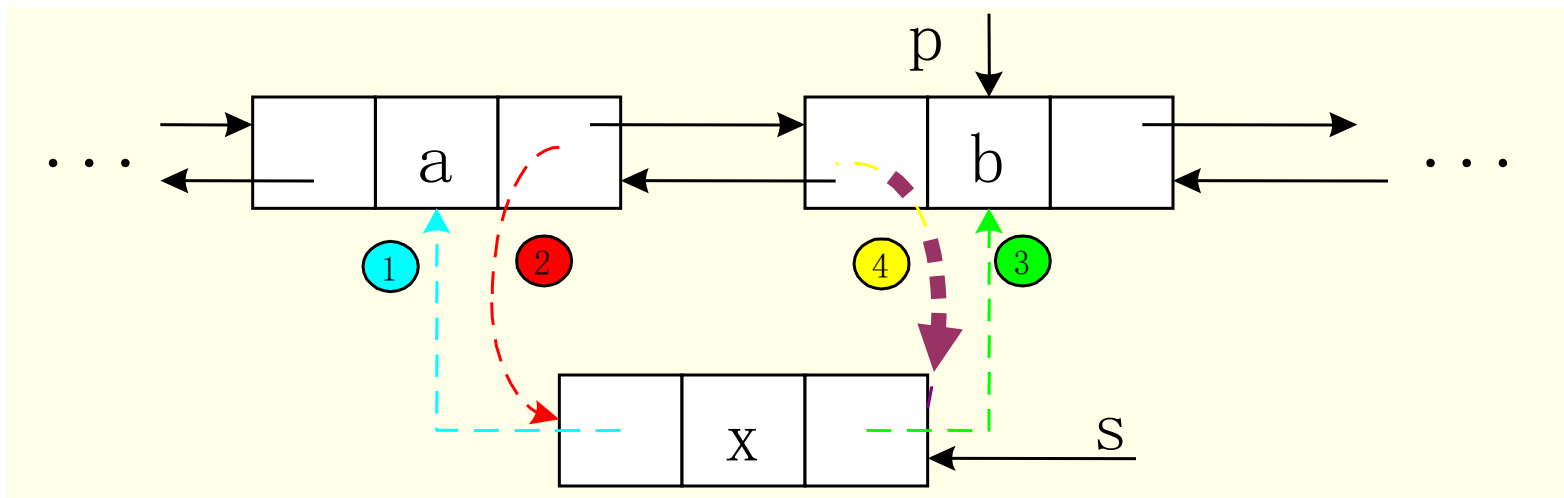


1.  $s \rightarrow \text{prior} = p \rightarrow \text{prior};$

2.  $p \rightarrow \text{prior} \rightarrow \text{next} = s;$

3.  $s \rightarrow \text{next} = p;$

# 双向链表的插入【算法2.13】 P39



1.  $s \rightarrow \text{prior} = p \rightarrow \text{prior};$

2.  $p \rightarrow \text{prior} \rightarrow \text{next} = s;$

3.  $s \rightarrow \text{next} = p;$

4.  $p \rightarrow \text{prior} = s;$

# 双向链表的插入 (把元素e插入到第i个位置)

**Status ListInsert\_DuL( DuLinkList &L, int i, ElemType e)**

**{ //确定表L中原本在第i个位置上的结点, 指针p指向它**

**if( ! (p=GetElemP\_DuL(L,i)) ) return ERROR;**

**s=new DuLNode; //生成新结点, 指针s指向它**

**s->data=e; //将结点s的数据域置为e**

**s->prior=p->prior;**

**p->prior->next=s;**

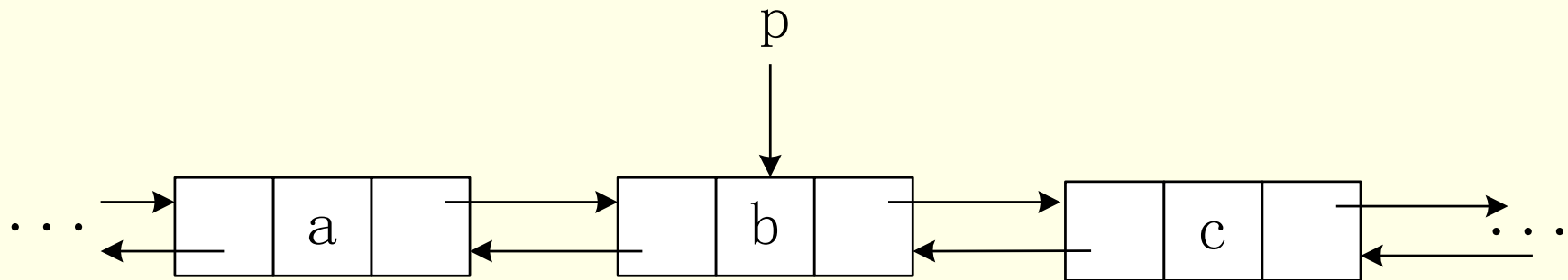
**s->next=p;**

**p->prior=s;**

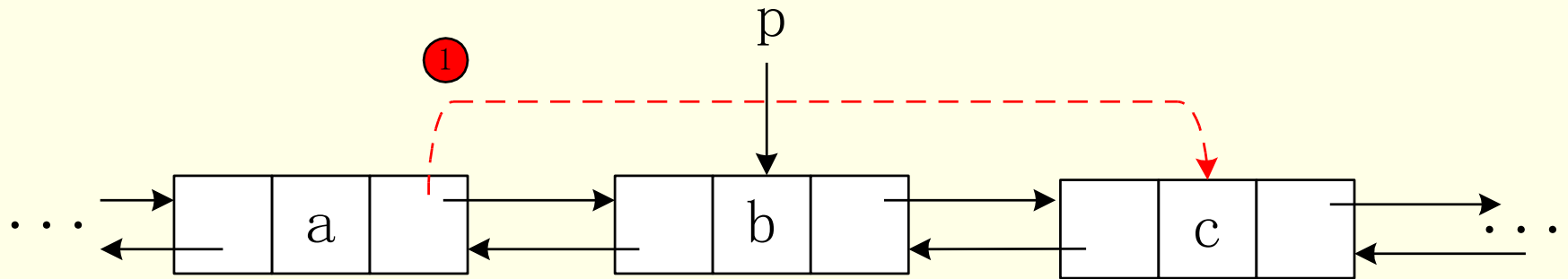
**return OK;**

**}**

# 双向链表的删除【算法2.14】 P40

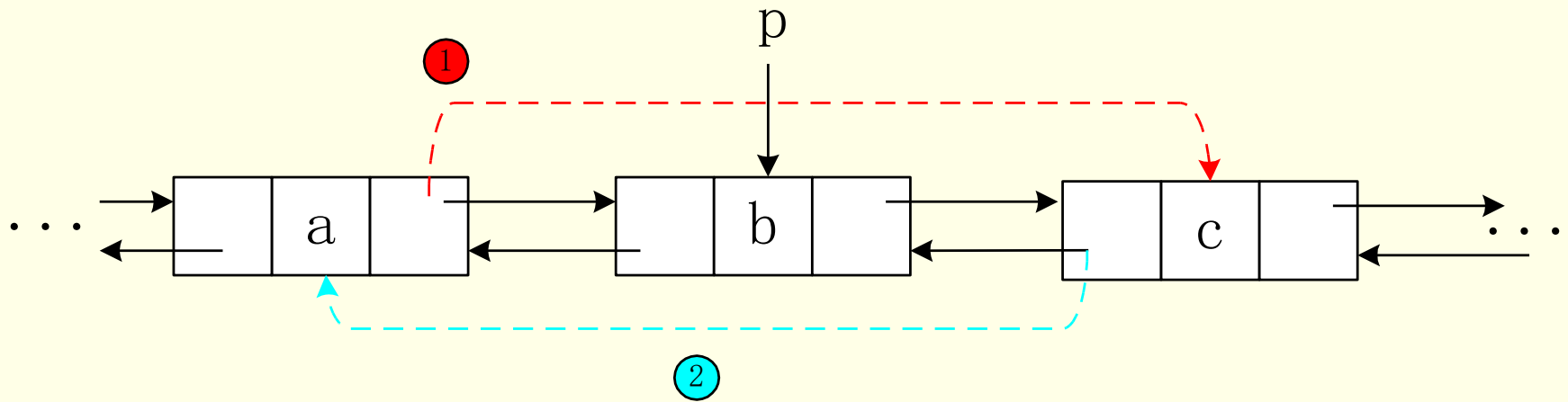


# 双向链表的删除【算法2.14】 P40



**1.  $p \rightarrow \text{prior} \rightarrow \text{next} = p \rightarrow \text{next};$**

# 双向链表的删除【算法2.14】 P40



**1.  $p \rightarrow \text{prior} \rightarrow \text{next} = p \rightarrow \text{next};$**

**2.  $p \rightarrow \text{next} \rightarrow \text{prior} = p \rightarrow \text{prior};$**

# 双向链表的删除（删除第i个结点）

**Status ListDelete\_DuL(DuLinkList &L,int i,ElemType &e)**

**{ //确定表L中原本在第i个位置上的结点，指针p指向它**

**if(!(p=GetElemP\_DuL(L,i)))      return ERROR;**

**e=p->data; //提前保存待删结点的数据域，赋给引用参数e**

**p->prior->next=p->next;**

**p->next->prior=p->prior;**

**delete p; //删除第i个结点，释放内存空间**

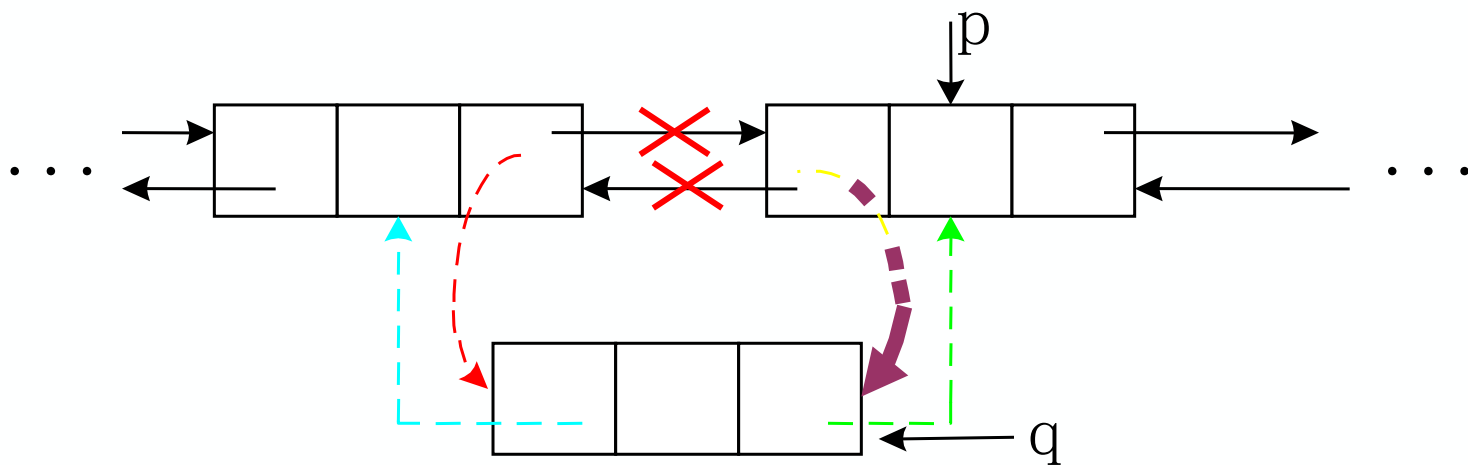
**return OK;**

**}**

在双链表中向p所指的结点之前插入一个结点q的操作为 ( )

- A.  $p \rightarrow \text{prior} = q; q \rightarrow \text{next} = p; p \rightarrow \text{prior} \rightarrow \text{next} = q; q \rightarrow \text{prior} = p \rightarrow \text{prior};$
- B.  $q \rightarrow \text{prior} = p \rightarrow \text{prior}; p \rightarrow \text{prior} \rightarrow \text{next} = q; q \rightarrow \text{next} = p; p \rightarrow \text{prior} = q \rightarrow \text{next};$
- C.  $q \rightarrow \text{next} = p; p \rightarrow \text{next} = q; q \rightarrow \text{prior} \rightarrow \text{next} = q; q \rightarrow \text{next} = p;$
- D.  $p \rightarrow \text{prior} \rightarrow \text{next} = q; q \rightarrow \text{next} = p; q \rightarrow \text{prior} = p \rightarrow \text{prior}; p \rightarrow \text{prior} = q;$

答案: D



## 【2016年计算机联考真题】

已知一个带有表头结点的双向循环链表L，结点结构为

|       |      |      |
|-------|------|------|
| prior | data | next |
|-------|------|------|

，其中prior和next分别为指向其直接前驱和直接后继结点的指针，现要删除指针p所指的结点，正确的语句序为（ ）。

- A.  $p \rightarrow \text{next} \rightarrow \text{prior} = p \rightarrow \text{prior}$ ;  $p \rightarrow \text{prior} \rightarrow \text{next} = p \rightarrow \text{prior}$ ;  $\text{free}(p)$ ;
- B.  $p \rightarrow \text{next} \rightarrow \text{prior} = p \rightarrow \text{next}$ ;  $p \rightarrow \text{prior} \rightarrow \text{next} = p \rightarrow \text{next}$ ;  $\text{free}(p)$ ;
- C.  $p \rightarrow \text{next} \rightarrow \text{prior} = p \rightarrow \text{next}$ ;  $p \rightarrow \text{prior} \rightarrow \text{next} = p \rightarrow \text{prior}$ ;  $\text{free}(p)$ ;
- D.  $p \rightarrow \text{next} \rightarrow \text{prior} = p \rightarrow \text{prior}$ ;  $p \rightarrow \text{prior} \rightarrow \text{next} = p \rightarrow \text{next}$ ;  $\text{free}(p)$ ;

**答案：D** 无论是链表的插入还是删除都必须保证不断链。

# 练习一下

1. 单链表不具备的特点是 (A) 。

- (A) 可随机访问任一结点      (B) 插入与删除不需要移动元素  
(C) 不必事先估计存储空间      (D) 所需空间与其长度成正比

2. 带头结点的单链表L为空的判断条件是 (B) 。

- (A)  $L == \text{NULL}$  不带头结点的单链表判空      (B)  $L \rightarrow \text{next} == \text{NULL}$   
(C)  $L \rightarrow \text{next} == L$  单循环链表判空      (D)  $L != \text{NULL}$

3. 与单链表相比，双链表的优点之一是 (D) 。

- (A) 插入删除更简单      (B) 可以进行随机访问  
(C) 可以省略表头指针或表尾指针  
(D) 顺序访问相邻结点更灵活

4. 判断：线性表的逻辑顺序与存储顺序总是一致的。

